

# $\mathbb{F}2\mathbb{Z}$

immediate

July 31, 2026

## Abstract

Albert: Should the name be  $\mathbb{F}2\mathbb{F}$ ?

## Contents

|          |                                                                                                             |           |
|----------|-------------------------------------------------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                                                                                         | <b>2</b>  |
| 1.1      | Acknowledgements . . . . .                                                                                  | 6         |
| <b>2</b> | <b>Preliminaries</b>                                                                                        | <b>6</b>  |
| <b>3</b> | <b><math>\mathbb{F}2\mathbb{Z}</math></b>                                                                   | <b>6</b>  |
| 3.1      | Constrained codes over dual fields . . . . .                                                                | 6         |
| 3.2      | Tensor decomposition, interleaved codes, and products of constrained codes                                  | 8         |
| 3.3      | The main $\mathbb{F}2\mathbb{Z}$ IOR . . . . .                                                              | 9         |
| 3.4      | An optimized GKR protocol for the $\mathbb{F}2\mathbb{Z}$ grand products . . . . .                          | 10        |
| 3.5      | Proof size optimization for large inputs . . . . .                                                          | 11        |
| 3.6      | An optimized GKR protocol for the $\mathbb{F}2\mathbb{Z}$ grand products . . . . .                          | 11        |
| 3.7      | Proof size optimization for large inputs . . . . .                                                          | 11        |
| <b>4</b> | <b>Virtualization of <math>\mathbb{F}_2</math>-linear combinations lifted to integers</b>                   | <b>12</b> |
| 4.1      | Optimizing $\mathbb{F}2\mathbb{Z}$ for virtualized inputs $\pi_2^{-1}(M \cdot \pi_2(\mathbf{f}))$ . . . . . | 15        |
| <b>5</b> | <b>R1CS constraints over polynomial rings with ideal membership</b>                                         | <b>15</b> |
| <b>6</b> | <b>Open questions for us</b>                                                                                | <b>16</b> |

|          |                                                 |           |
|----------|-------------------------------------------------|-----------|
| <b>7</b> | <b>Case-study</b>                               | <b>16</b> |
| 7.1      | Integer and modular multiplication . . . . .    | 16        |
| 7.2      | SHA-256 hashing . . . . .                       | 16        |
| 7.3      | Experiments . . . . .                           | 18        |
| 7.3.1    | Comparison with Hyrax . . . . .                 | 18        |
| 7.3.2    | Comparison with Flock . . . . .                 | 18        |
| <b>A</b> | <b>Ring switching via Galois orbits</b>         | <b>18</b> |
| <b>B</b> | <b>Bilinear Embeddings</b>                      | <b>19</b> |
| B.1      | Background . . . . .                            | 20        |
| B.1.1    | Perfect pairings . . . . .                      | 20        |
| B.1.2    | Succinct vectors . . . . .                      | 20        |
| B.2      | Structure . . . . .                             | 21        |
| B.3      | Constructions . . . . .                         | 24        |
| B.3.1    | Trace embedding . . . . .                       | 24        |
| B.3.2    | Coefficient projection . . . . .                | 25        |
| B.4      | Extension openings . . . . .                    | 26        |
| B.4.1    | Batching protocol . . . . .                     | 26        |
| B.4.2    | Folding protocol . . . . .                      | 27        |
| <b>C</b> | <b>GKR protocol for multiple grand products</b> | <b>34</b> |

## 1 Introduction

Why does this paper exist? F2Z has many similarities to Binius64, so why F2Z at all? First of all, Binius64 is an end-to-end proving system, while F2Z is a PCS. Let’s consider F2Z-SNARK a SNARK for proving R1CS constraints over  $\mathbb{Z}$  or  $\mathbb{Z}[X]$  with ideals. Flock is also related to F2Z-SNARK, though less so, since Flock is a more use-case specific SNARK for proving bit-intensive computations. We should also keep in mind that Flock is, at the end of the day, Binius64 on steroid optimizations. In general, I think of Flock as subsumed by a Binius64 repo that adopts Flock’s optimizations. Hence I will not discuss Flock too much here.

In what follows I understand F2Z-SNARK to be a proof system for constraints over  $\mathbb{Z}, \mathbb{Z}[X]$  with ideals, binary fields, etc., like Zinc+ at the arithmetization and PIOP level (applied to R1CS-like constraints), but using F2Z as PCS.

**Efficiency and costs** I will now compare F2Z-SNARK with Binius64 on a operation-by-operation case:

- *Integer and modular addition.* F2Z-SNARK can handle addition or modular addition of arbitrarily sized integers. Even if the integers are gigantic, the projection to random field trick compresses their size to 100-ish bits.

On the other hand, integer addition or modular addition (how does Binius handle modular addition when the modulus is not  $2^n$  actually?) in Binius64 requires committing to the carry bits (so each single addition adds a new 32-64 bit word to the witness), and cannot handle large integer/modular addition without chunking.

Further, addition in Binius64 produces a non-linear constraint, because proving that the committed carry bits are indeed carry bits requires a Hadamard product. In F2Z-SNARK, additions are linear constraints.

- *Integer or modular multiplication.* This is a quadratic constraint in F2Z-SNARK. It does not introduce extra witness<sup>1</sup>. Again, it supports arbitrary sizes and benefits from the random projection for large sizes.

Binius64 supports multiplication of elements of less than 32 bits. Larger sizes require chunking. Under 32 bits, it does not pay much for multiplication at the constraint level. The way it proves the multiplications is via a grand product similar to F2Z-SNARK grand product in MLE evaluations.

A computation where we are just making  $m$  multiplications of under 32 bit integers may actually end up costing almost the same between Binius64 and F2Z-SNARK.

[Albert: Let's run the experiment](#)

It is interesting that in Binius64, multiplications don't go through the Spartan-like PIOP, and instead are proved directly with the grand product [Albert: At least that's my understanding](#). In F2Z-SNARK, multiplications would indeed go through the normal PIOP pipeline. Let's keep in mind as something F2Z-SNARK could mimic.

[Albert: Interesting](#)

- *XOR*
- *AND*
- *Proof system costs*

**Specific performance** This is WIP, but when it comes to hash proving, F2Z-SNARK seems to outperform in all axes Binius/Flock when there are less than  $2^8$  hashes to prove. Beyond that, the F2Z-SNARK route points to 1.5-3x prover times, comparable proof sizes, significantly smaller RAM usage, and unknown verifier comparison for now [Albert: WIP](#).

---

<sup>1</sup>Or it negligible does, because of the possible  $+n \cdot w$  term in a modular equation, where  $n$  is the modulus and  $w$  is a committed integer –very small in bit size in practice

The practical max number of hashes that F2Z-SNARK can handle is higher than in Flock due to the lesser memory usage.

The fact that F2Z-SNARK performs better for smaller number of hashes may be due to F2Z-SNARK working with a smaller witness. At those sizes, there's likely floor costs (e.g. Merkle openings) that somewhat dominate costs, and having a 2x (roughly) smaller witness may start to be relevant.

Interesting to note that in many use cases (e.g. digital Id) one has less than  $2^8$  hashes to prove. Flock-like numbers come up in Ethereum signature aggregation and maybe proof of provenance where one has to prove a hash of a large file (and others?).

[Albert: Current WIP: hash + ECDSA sign comparison.](#) [Quantized ML linear layer](#)

For initial reference, Binius64 takes about 110ms to prove an ECDSA with a 200KB proof size.

**Scientific contributions** So far, we have compared Binius64 and F2Z-SNARK in terms of performance. But how about the scientific contributions of F2Z-SNARK?

F2Z-SNARK is a proof system designed to handle constraints over  $\mathbb{Z}$ ,  $\mathbb{Z}[X]$  with ideal membership, binary fields, and other fields, simultaneously (in v3 we are probably sticking to integers and  $\mathbb{Z}[X]$  for simplicity). Arithmetization and PIOP-wise, it borrows from Zinc+, and the novelty compared to Zinc+ is that we are committing to the integer bits over a binary field. For bit-intensive computations, this is much better than committing over  $\mathbb{Z}$ . It remains to be seen what happens in integer-heavy computations [Albert: TODO! Edit: John Wu already compared it to Hyrax.](#)

Potentially, we could commit to large integers with a different PCS over  $\mathbb{Z}$ , say with IPRS, and create a recursive proof of opening for the two PCS's, using F2Z since PCS opening proofs are hash heavy.

Thus some scientific contributions of F2Z-SNARK are

- Unlocks good scaling of bit intensive computations in Zinc+-like proof systems.
- If we include Zinc+ in this line of work, F2Z-SNARK is a follow up for a proof system that uses a highly expressive constraint system. Due to the expressivity, the use-cases of F2Z-SNARK are potentially huge, and difficult to predict. For example, the F2Z-PCS could be used as a drop-in replacement for elliptic curve PCS's (as shown by John Wu)

We (tentatively) choose F2Z-SNARK (likely with both WHIZ and F2Z) working exclusively on  $\mathbb{Z}$  or  $\mathbb{Z}[X]$  as our route for a general zkDSL, and the motivation was the expressivity of the constraints.

- F2Z-SNARK does not quite provide much novelty regarding SNARKs over binary fields, beyond possibly some optimizations in the grand product proof (e.g. lower levels of the grand product GKR are computed via a lookup table, exploiting the limited options we have in leafs due to the bit-ness of the witness).

|                         | F2Z-PCS            | Zinc+-PIOP     | Binius64      | Flock         | Plonky3         | Zinc+ |
|-------------------------|--------------------|----------------|---------------|---------------|-----------------|-------|
| ECDSA                   |                    |                | 119/6.3/195/? | NA            |                 |       |
| RSA                     |                    |                |               | NA            |                 |       |
| $2^{17}$ int mul 16bits | 48/2/80/62         |                | 217/9/313/?   | NA            | 367/7/340/?     |       |
| $2^{17}$ int mul 32bits | 74/2.4/86/124      |                | 218/9/313/?   | NA            | 571/7.6/373/?   |       |
| 1M int mul 16bits       | 192/2.9/112/483    |                | 3600/61/440/? | NA            | 2920/9.8/492/?  |       |
| 1M int mul 32bits       | 266/3.7/119/863    |                | 3800/62/440/? | NA            | 5050/10.2/520/? |       |
| $2^7$ SHA               | 12.7/1.75/129/14*  | expected negl. | 37/3.4/249/?  | 33/25/112/?   |                 |       |
| $2^{11}$ SHA            | 93.5/2.8/109/197*  | expected negl. | 126/27/381/?  | 47.2/27/172/? |                 |       |
| $2^{15}$ SHA            | 1320/4.8/170/3080* | expected negl. | Stuck         | 425/25/233/?  |                 |       |
| $2^8$ Blake3            |                    | expected negl. | 31/2.6/249/?  | 256/254/111/? |                 |       |
| $2^{15}$ Blake3         |                    | expected negl. | Stuck         | 386/267/160/? |                 |       |

**Table 1:** \*= we are not using virtualization of XOR. The committed and opened witness size is 4x larger than it could be. [Albert](#): Missing costs for F2Z PIOP. Note these are negligible for the hash benches though, and likely quite less than the grand product cost in other benches. All these were obtained asking AI to run benches. For the final paper we should make sure we are doing fair and correct benching (e.g. I’m not sure memory consumption is measured the same way across benches, especially for Binius, which is max RSS), but this helps get a rough idea how F2Z performs relatively to other systems. Prover\_time(ms)/Verifier\_time(ms)/Proof\_size(KB)/Peak\_memory(MB). Integer mult measure one million u16 x u16 -> u32 mults. F2Z-PCS does not include PIOP costs – these are negligible in the hash benches (linear constraints), but should be accounted for in the other benches (degree 2 constraints). Ran in MacBook Air M4 16GB.

Flock’s verifier seems a bit broken in general. Flock’s Blake3 prover for small number of hashes seems broken too. I modified Flock’s Ligerito config so that it produces smaller proofs at very little extra cost. For Blake3 we couldn’t do  $2^7$  hashes because Flock errors out on  $2^7$  hashes [Albert](#): [todo: make SHA also do  \$2^8\$  instead of  \$2^7\$](#)

F2Z-PCS hash performance was “proxied” as follows: for SHA-256,  $2^{14+k}$  witness corresponds to  $2^k$  hashes. Better arithmetization may be possible. We [\\*DID NOT\\*](#) leverage virtualization of XOR, which allows to run Ligerito for  $2^{12+k}$  witness size, rather than  $2^{14+k}$  (The GKR grand product still would use  $2^{14+k}$  factors). [Albert](#): [VERY IMPORTANT: It is possible that the GKR grand product can be optimized in this setting. Right now we are just proving a  \$2^{14+k}\$  on unstructured bits, but in reality many of these bits are linear combinations in  \$\mathbb{F}\_2\$  of witness bits.](#)

Binius64 achieves the highest number of mults per second at  $2^{17}$  mults. Binius64 at multiplication of 16bit integers pays a penalty because the system uses 64-bit words by default, so it either uses a 64 bits per 16bit witness element and range check the word, or pack different 16 and 32 bit integers into the same 64 bit word, but then it has to relate the product of the 64 bit words to the products of the original words (or unpack them as 64 bit words before multiplying). OTOH doing more Binius-native mults ( $32 \times 32 \rightarrow 64$  bits) does not perform amazingly either [Albert](#): [This most likely heavily disproves my belief that Binius64 and F2Z-SNARK are similar in integer multiplication. This is a good showcase of the flexibility of arithmetizing the way we do: we don’t need to use predefined word sizes: here our R1CS is just reconstruction of integers via linear combinations with powers of twos, and Hadamard product \(note that we are not measuring the cost of the PIOP in F2Z! But it should be cheaper than the grand product, and amenable to some better optimization \(univariate skip e.g.\). But in any case, I’m not even sure the issue is the rigidity of the arithmetization in Binius64, since  \$32 \times 32 \rightarrow 64\$  is Binius friendly as far as I understand, and performance is mid](#)

That being said, some of these optimizations could be published as contributions, following Flock’s lead. [Albert: Sometimes I undersell my work, this is a memo to myself to be mindful with that](#)

There’s also the virtualization of XOR elements of F2Z-PCS. In a sense, a similar thing occurs in Binius64: their R1CS constraint system admits hadamard products, replacing the hadamard by integer multiplication. So such constraint is multiplying together linear combinations of the witness (over the binar field, so “XOR combinations”).

Our virtualization trick can be understood as if we had an R1CS over  $\mathbb{Z}$ , with witness  $\mathbf{z}$ , and an extra constraint  $\mathbf{z} = \pi_2^{-1}(M\pi_2(\mathbf{z}))$  for some matrix  $M$ . What Binius is doing can be understood similarly, but with much more limited expressivity. Hence in this sense Binius64 arithmetization is a specific case of our arithmetization.

[Albert: Noting so I don’t forget: the idea of F2Z being a drop-in replacement alternative to Hyrax \(PQ secure and more efficient\) may be a good extra motivating point](#)

## 1.1 Acknowledgements

## 2 Preliminaries

Given a prime  $q$  we let  $\pi_q : \mathbb{Z} \rightarrow \mathbb{F}_q$  be the ring homomorphism mapping an integer to its reduction modulo  $q$ . We further let  $\pi_0$  denote the identity map on  $\mathbb{Z}$ , so that  $\pi_0(x) = x$  for all  $x \in \mathbb{Z}$ . We let  $\mathbb{F}_0 = \mathbb{Z}$ .

Given a finite field  $\mathbb{F}$  of characteristic  $q$ , let  $a(X) \in \mathbb{Z}[X]$  be an irreducible polynomial over  $\mathbb{F}_q$ . [Albert: yes?](#), so that  $\mathbb{F} = \mathbb{Z}[X]/(q, a(X))$ . [Albert: consider isomorphism, or assume all finite fields are represented like this.](#) Let  $\psi_{\mathbb{F}} : \mathbb{Z}[X] \rightarrow \mathbb{F}$  be the canonical homomorphism from  $\mathbb{Z}[X]$  to  $\mathbb{F}$  obtained by reducing modulo the ideal  $(q, a(X))$ , i.e. by reducing coefficients modulo  $q$  and then reducing the resulting polynomial modulo  $a(X)$ .

**Interactive Oracle Proofs of Proximity (IOPP)**   Interleaved codes

**Interactive Oracle Reductions (IOR)**

## 3 F2Z

### 3.1 Constrained codes over dual fields

[Albert: not sure dual is a good word: the two fields are just “different”, while dual things tend to have a close relation between each other](#) Let  $\ell_0 \geq 0$  be a dimension parameter, and  $m \geq 0$  a blocklength parameter. Let  $\mathbb{F}$  be a field (not necessarily a prime field), and let

$\mathcal{C} \subseteq \mathbb{F}^m$  be a linear code of dimension  $\ell_0$  and blocklength  $m$  over  $\mathbb{F}$ . Let  $k \geq 0$  be a public input length parameter, and let  $Q \in \mathbb{F}[Y_1, \dots, Y_k, Z_1, \dots, Z_\ell]$  be a constraint polynomial, of arbitrary degree, with coefficients in  $\mathbb{F}$  and on  $k + \ell$  variables. Further, let  $\ell \geq \ell_0$  be a witness length parameter, and let  $P : \mathbb{F}^\ell \rightarrow \mathbb{F}^{\ell_0}$  be an arbitrary map (looking ahead, for us  $P$  will be a packing map that packs elements from  $\mathbb{F}_2$  into elements of a larger binary field. Consider the following relation:

$$\mathcal{R}_{\mathcal{C}, \mathbb{F}, P}^Q := \left\{ \left( \begin{array}{l} \mathbf{x} = \mathbf{y} \in \mathbb{F}^k, \\ \mathbf{y} = f \in \mathbb{F}^m, \\ \mathbf{w} = \mathbf{f} \in \mathbb{F}^\ell \end{array} \right) \middle| \begin{array}{l} \text{Enc}_{\mathcal{C}}(P(\mathbf{f})) = f \\ \wedge Q(\mathbf{y}, \mathbf{f}) = 0 \end{array} \right\}$$

When  $Q$  and  $\mathbf{y}$  are such that  $\mathbf{y} = (\mathbf{v}, \mu) \in \mathbb{F}^\ell \times \mathbb{F}$  and

$$Q(\mathbf{y}, \mathbf{f}) = \langle \mathbf{v}, \mathbf{f} \rangle = \sum_{i \in [\ell]} \mathbf{v}_i \cdot \mathbf{f}_i,$$

we denote  $\mathcal{R}_{\mathcal{C}, \mathbb{F}, P}^Q$  by  $\mathcal{R}_{\mathcal{C}, \mathbb{F}, P}^{\text{lin}}$ .

Next let  $\mathbb{F}_{2^n}$  be a finite field of characteristic 2, and let  $\mathbb{F}_q$  be a finite field of prime order  $q$  or  $\mathbb{Q}$ . For intuition,  $\mathbb{F}_{2^n}$  is the field over which we will commit our witness, and  $\mathbb{F}_q$  is the field over which we prove linear relations on the witness. We let

$$\pi_2 : \mathbb{Z} \rightarrow \mathbb{F}_2 \subseteq \mathbb{F}_{2^n}, \quad \pi_q : \mathbb{Z} \rightarrow \mathbb{F}_q,$$

be reduction mod 2 and  $q$  maps. We further let

$$\pi_2^{-1} : \mathbb{F}_2 \rightarrow \{0, 1\} \subseteq \mathbb{Z}, \quad \pi_q^{-1} : \mathbb{F}_q \rightarrow [0 .. q - 1] \subseteq \mathbb{Z},$$

be maps such that  $\pi_2(\pi_2^{-1}(x)) = x$  and  $\pi_q(\pi_q^{-1}(x)) = x$  for all  $x$ . We remark that one could choose any other such “inverse” maps, e.g. for  $\pi_q^{-1}$  one may prefer to use centered representatives.

Assume  $|\mathbb{F}_{2^n}| = 2^n$ . Let

$$\beta_1, \dots, \beta_n \in \mathbb{F}_{2^n}$$

be an  $\mathbb{F}_2$ -base of  $\mathbb{F}_{2^n}$  seen as an  $\mathbb{F}_2$ -vector space of dimension  $n$ .

Given a vector  $\mathbf{u} \in \mathbb{F}_{2^n}^m$ , with  $2^n$  dividing  $m$ , we define

$$\text{Pack}_n(\mathbf{u}) \in \mathbb{F}_{2^n}^{m/n}$$

as the vector whose  $j$ -th entry is

$$\text{Pack}_n(\mathbf{u})_j = \mathbf{u}_{(j-1) \cdot n + 1} \cdot \beta_1 + \dots + \mathbf{u}_{(j-1) \cdot n + n - 1} \cdot \beta_{n-1} + \mathbf{u}_{j \cdot n} \cdot \beta_n \in \mathbb{F}_{2^n}.$$

Let  $\mathcal{C} \subseteq \mathbb{F}_{2^n}^m$  be a linear code of dimension  $\ell/2^n$  and blocklength  $m$  over  $\mathbb{F}_{2^n}$ . Define

$$\mathcal{R}_{\mathcal{C}, \mathbb{F}_{2^n}, \mathbb{F}_q, \text{Pack}_n}^{\text{lin-dual}} := \left\{ \left( \begin{array}{l} \mathbf{x} = (\mu, \mathbf{v}) \in \mathbb{F}_q \times \mathbb{F}_q^\ell, \\ \mathbf{y} = f \in \mathbb{F}_{2^n}^m, \\ \mathbf{w} = \mathbf{f} \in \{0, 1\}^\ell \subseteq \mathbb{Z}^\ell \end{array} \right) \middle| \begin{array}{l} \text{Enc}_{\mathcal{C}}(\text{Pack}_n(\pi_2(\mathbf{f}))) = f \\ \wedge \langle \pi_q(\mathbf{f}), \mathbf{v} \rangle = \mu \end{array} \right\}$$

### 3.2 Tensor decomposition, interleaved codes, and products of constrained codes

We fix a tensor decomposition of vectors  $\mathbf{v} \in \mathbb{F}_{2^n}^\ell$  as

$$\mathbf{v} = \mathbf{v}^{(1)} \otimes \mathbf{v}^{(2)} = \left( \mathbf{v}_i^{(1)} \cdot \mathbf{v}_j^{(2)} \right)_{i \in [\ell_1], j \in [\ell_2]}$$

for some  $\mathbf{v}^{(1)} \in \mathbb{F}_q^{\ell_1}, \mathbf{v}^{(2)} \in \mathbb{F}_q^{\ell_2}$ , with  $\ell_1 \cdot \ell_2 = \ell$ . Note that the trivial tensor decomposition is admitted, i.e. we may effectively not decompose  $\mathbf{v}$  by choosing  $\ell_1 = \ell, \ell_2 = 1, \mathbf{v}^{(1)} = \mathbf{v}$ , and  $\mathbf{v}^{(2)} = (1)$ .

We assume  $\mathcal{C} = \mathcal{C}_1^{\equiv \ell_2}$  is the  $\ell_2$ -dimensional interleaving of a code  $\mathcal{C}_1 \subseteq \mathbb{F}_{2^n}^{m/\ell_2}$  of dimension  $\ell_1/2^n$  and blocklength  $m/\ell_2$ . We then index the entries of  $\mathbf{f}$  as  $\mathbf{f} = (\mathbf{f}_{ij})_{i \in [\ell_1], j \in [\ell_2]}$ . Note that

$$\langle \pi_q(\mathbf{f}), \mathbf{v} \rangle = \sum_{j \in [\ell_2]} \left( \sum_{i \in [\ell_1]} \pi_q(\mathbf{f}_{ij}) \cdot \mathbf{v}_i^{(1)} \right) \cdot \mathbf{v}_j^{(2)} = \sum_{j \in [\ell_2]} \langle \pi_q(\mathbf{f}_{*j}), \mathbf{v}^{(1)} \rangle \cdot \mathbf{v}_j^{(2)},$$

where  $\mathbf{f}_{*j} = (\mathbf{f}_{ij})_{i \in [\ell_1]}$ .

Define the following bivariate polynomial:

$$P(Y, Z) = (Y - 1) \cdot Z + 1$$

Note  $P(Y, 0) = 1$  and  $P(Y, 1) = Y$ . Consider the relation  $\mathcal{R}_{\mathbb{F}_{2^n}, \text{Pack}_n}^{Q_{\mathbb{F}2\mathbb{Z}}}$  with underlying code  $\mathcal{C}_1$  and

$$Q_{\mathbb{F}2\mathbb{Z}}(Y_1, \dots, Y_{\ell_1+1}, Z_1, \dots, Z_{\ell_1}) := \prod_{i=1}^{\ell_1} P(Y_i, Z_i) - Y_{\ell_1+1}.$$

Precisely,

$$\mathcal{R}_{\mathcal{C}_1, \mathbb{F}_{2^n}, \text{Pack}_n}^{Q_{\mathbb{F}2\mathbb{Z}}} = \left\{ \left( \begin{array}{l} \mathbf{x} = (\mathbf{y}, z) \in \mathbb{F}_{2^n}^{\ell_1} \times \mathbb{F}_{2^n}, \\ \mathbf{y} = f \in \mathbb{F}_{2^n}^{m/\ell_2}, \\ \mathbf{w} = \mathbf{f} \in \mathbb{F}_{2^n}^{\ell_1} \end{array} \right) \left| \begin{array}{l} \text{Enc}_{\mathcal{C}_1}(\text{Pack}_n(\mathbf{f})) = f \\ \wedge \prod_{i=1}^{\ell_1} ((\mathbf{y}_i - 1) \cdot \mathbf{f}_i + 1) = z \end{array} \right. \right\}$$

We assume prover and verifier have access to an IOR  $\Pi_{\text{red}}$  that reduces  $\mathcal{R}_{\mathcal{C}_1, \mathbb{F}_{2^n}, \text{Pack}_n}^{Q_{\mathbb{F}2\mathbb{Z}}}$  to  $\mathcal{R}_{\mathcal{C}_1, \mathbb{F}_{2^n}, \text{Pack}_n}^{\text{lin}}$ . In our instantiation  $\Pi_{\text{red}}$  is an optimized GKR protocol for grand products, which reduces the initial grand product claim to MLE evaluation claims of  $\mathbf{y}$  and  $\mathbf{f}$  over  $\mathbb{F}_{2^n}$ . As we explain in ??, we design several optimizations for the specific shape of our grand products.

We assume  $\Pi_{\text{red}}$  works so that the verifier never queries  $\mathbf{y}$ . Note that this is indeed the case with GKR (indeed, GKR reduces the initial circuit evaluation claim on the input and outputs, into MLE evaluation claims of the input and outputs, without querying any oracle in-between).



Further, we consider the  $\ell_2$ -fold parallel composition  $\Pi_{\text{red}}^{\ell_2}$  of  $\Pi_{\text{red}}$ , and we assume that given  $\ell_2$  triples

$$\left( (\mathbf{y}^{(j)}, \mu^{(j)}), g^{(j)}, \mathbf{h}^{(j)} \right)_{j \in [\ell_2]} \in \left( \mathcal{R}_{\mathcal{C}_1, \mathbb{F}_{2^n}, \text{Pack}_n}^{Q_{\mathbb{F}2\mathbb{Z}}} \right)^{\ell_2}$$

the IOR  $\Pi_{\text{red}}^{\ell_2}$  outputs

$$\left( (\mathbf{v}, \mu_j) \in \mathbb{F}_{2^n}^{\ell_1} \times \mathbb{F}_{2^n}, g^{(j)}, \mathbf{h}^{(j)} \right)_{j \in [\ell_2]} \in \left( \mathcal{R}_{\mathcal{C}_1, \mathbb{F}_{2^n}, \text{Pack}_n}^{\text{lin}} \right)^{\ell_2},$$

for some  $\mathbf{r} \in \mathbb{F}_{2^n}^{\log \ell_1}$  and some  $\mu_j \in \mathbb{F}_{2^n}$  for  $j \in [\ell_2]$ . In words,  $\Pi_{\text{red}}^{\ell_2}$  outputs  $\ell_2$  linear claims for the input witnesses  $\mathbf{h}^{(j)}$  behind the input oracles  $g^{(j)}$ , all *on the same vector of coefficients*  $\mathbf{v}$ . In our instantiation  $\Pi_{\text{red}}^{\ell_2}$  is simply the GKR protocol applied to several grand product claims at once, with optimizations discussed in ??.

### 3.3 The main $\mathbb{F}2\mathbb{Z}$ IOR

We provide an IOPP for  $\mathcal{R}_{\mathcal{C}, \mathbb{F}_{2^n}, \mathbb{F}_q, \text{Pack}_n}^{\text{lin-dual}}$  next.

**Construction 3.1** (IOPP for  $\mathcal{R}_{\mathcal{C}, \mathbb{F}_{2^n}, \mathbb{F}_q, \text{Pack}_n}^{\text{lin-dual}}$ ). The prover and verifier receive  $\mathbf{x} = (\mu, \mathbf{v}) \in \mathbb{F}_q \times \mathbb{F}_q^\ell$  and  $\mathbf{y} = f \in \mathbb{F}_{2^n}^m$ . The prover receives  $\mathbf{w} = \mathbf{f} \in \{0, 1\}^\ell \subseteq \mathbb{Z}^\ell$  such that  $(\mathbf{x}, \mathbf{y}, \mathbf{w}) \in \mathcal{R}_{\mathcal{C}, \mathbb{F}_{2^n}, \mathbb{F}_q, \text{Pack}_n}^{\text{lin-dual}}$ . We assume the tensor split from the previous Section ??, and adopt its notation. Let  $g \in \mathbb{F}_{2^n}$  be a nonzero element from  $\mathbb{F}_{2^n}$  with sufficiently large multiplicative order, to be determined later.

1. The prover sends  $\mu_1, \dots, \mu_{\ell_2} \in \mathbb{Z}$  such that  $\langle \mathbf{f}_{*j}, \pi_q^{-1}(\mathbf{v}^{(1)}) \rangle = \mu_j$ .
2. The verifier checks that

$$\sum_{j \in [\ell_2]} \pi_q(\mu_j) \cdot \mathbf{v}_j^{(2)} = \mu \quad \text{over } \mathbb{F}_q. \quad (1)$$

In ?? we describe a variation in which, instead of sending the elements  $\mu_1, \dots, \mu_{\ell_2}$ , the prover commits to their bit-decomposition over  $\mathbb{F}_{2^n}$ , and proves (1) the present IOR. This saves on proof size and verifier time when  $\ell$  is large.

3. Prover and verifier define  $\gamma_i = \pi_q^{-1}(\mathbf{v}_i^{(1)}) \in [0 .. q - 1] \subseteq \mathbb{Z}$  for  $i \in [\ell_1]$ , and set to prove the following claim:

$$\prod_{i \in [\ell_1]} g^{\gamma_i \cdot \mathbf{f}_{ij}} = g^{\mu_j} \in \mathbb{F}_{2^n}, \quad \text{for all } j \in [\ell_2].$$

Note that this claim is equivalent to

$$\prod_{i \in [\ell_1]} ((g^{\gamma_i} - 1) \cdot \pi_2(\mathbf{f}_{ij}) + 1) = g^{\mu_j} \in \mathbb{F}_{2^n}, \quad \text{for all } j \in [\ell_2].$$

Notice that  $(\mathbb{x}_j = ((g^{\gamma_i})_{i \in [\ell_1]}, g^{\mu_j}), \mathbb{y}_j = f_{*j}, \mathbb{w}_j = \pi_2(\mathbf{f}_{*j})) \in \mathcal{R}_{\mathcal{C}_1, \mathbb{F}_{2^n}, \text{Pack}_n}^{Q_{\mathbb{F}2\mathbb{Z}}}$  for all  $j \in [\ell_2]$ . To prove the above claim, prover and verifier engage in the IOR  $\Pi_{\text{red}}^{\ell_2}$ <sup>2</sup> with input

$$(\mathbb{x}_j = ((g^{\gamma_i})_{i \in [\ell_1]}, g^{\mu_j}), \mathbb{y}_j = f_{*j}, \mathbb{w}_j = \pi_2(\mathbf{f}_{*j}))_{j \in [\ell_2]},$$

where in the honest case  $f_{*j} = \text{Enc}_{\mathcal{C}_1}(\text{Pack}_n(\pi_2(\mathbf{f}_{*j}))) \in \mathbb{F}_{2^n}^{m/\ell_2}$ . At the end of this IOR, they are left with the following claim to prove:

$$((\mathbf{v}, \mu'_j), f_{*j}, \pi_2(\mathbf{f}_{*j}))_{j \in [\ell_2]} \in \left( \mathcal{R}_{\mathcal{C}_1, \mathbb{F}_{2^n}, \text{Pack}_n}^{\text{lin}} \right)^{\ell_2} \quad (2)$$

for some vector  $\mathbf{v}$  and field elements  $\mu'_j$  output during the IOR  $\Pi_{\text{red}}^{\ell_2}$ .

4. Prover and verifier engage in a IOPP to certify the claims (2).

**Theorem 3.2.** *Albert: Completeness and RBR knowledge soundness of Construction 3.1*

*Proof.* It follows from Theorem 4.3 by taking  $\ell'_1 = \ell_1$  and taking  $M$  to be the identity matrix.  $\square$

We now describe a GKR-based IOR reducing  $(\mathcal{R}_{\mathcal{C}_1, \mathbb{F}_{2^n}}^Q)^{\ell_2}$  to  $(\mathcal{R}_{\mathcal{C}_1, \mathbb{F}_{2^n}}^{\text{eval}})^{\ell_2}$  of the required form.

**Construction 3.3** (Grand product GKR). *Albert: Todo: it's a standard grand product GKR so far – though there's lots of implementation-level optimizations it can undergo*

*Ulrich: Maybe to just write verbally, that we will use GKR for the grand product. And defer the explicit protocol description to the appendix or any later section? At this point it is maybe more important to clarify how useful the construction is because of packed commitment scheme over  $\mathbb{F}_2$ . Albert: Sounds good*

### 3.4 An optimized GKR protocol for the $\mathbb{F}2\mathbb{Z}$ grand products

The bulk of the work in  $\mathbb{F}2\mathbb{Z}$  (Construction 3.1) is to reduce the claim

$$\prod_{i \in [\ell_1]} ((g^{\gamma_i} - 1) \cdot \pi_2(\mathbf{f}_{ij}) + 1) = g^{\mu_j} \quad \in \mathbb{F}_{2^n}, \quad \text{for all } j \in [\ell_2]. \quad (3)$$

to  $\ell_2$  inner product claims for the vectors  $\pi_2(\mathbf{f}_{*j})$ . We do this reduction via the GKR protocol for multiple grand products, and make several implementation-level optimizations for it.

*Albert: Todo*

---

<sup>2</sup>Recall that, in our instantiation,  $\Pi_{\text{red}}^{\ell_2}$  is the GKR protocol applied to multiple grand products, cf. ??

### 3.5 Proof size optimization for large inputs

When  $\ell = \ell_1 \cdot \ell_2$  is such that  $\ell_2$  is large, say  $\ell \geq 2^{11}$ , the elements  $\mu_1, \dots, \mu_{\ell_2} \in \mathbb{Z}$  sent at Step 1 of [Construction 3.1](#) may start to dominate the proof size. For example, for  $\ell = 2^{13}$  and  $\mu_i$  of 128 bits, the elements  $\mu_1, \dots, \mu_{\ell_2}$  amount to 131 KB. To mitigate this cost, the idea is to have the prover commit to the vector  $(\mu_1, \dots, \mu_{\ell_2})$  and then prove (1), i.e.

$$\sum_{j \in [\ell_1]} \pi_q(\mu_j) \cdot \mathbf{v}_j^{(2)} = \mu \quad \text{over } \mathbb{F}_q.$$

using an IOPP for constrained codes. Concretely, the prover commits to the vector  $\text{bits}(\mu)$  of bits forming the elements  $(\mu_1, \dots, \mu_{\ell_2})$  using  $\mathbb{F}2\mathbb{Z}$ , and then proves (1) using [Construction 3.1](#). At the end of [Construction 3.1](#), the prover is left with an inner product claim for the vector  $\pi_2(\text{bits}(\mu))$ . [Albert: TODO: explain how to batch this claim with the WHIR IOPP for the final step of Construction 3.1](#)

### 3.6 An optimized GKR protocol for the $\mathbb{F}2\mathbb{Z}$ grand products

The bulk of the work in  $\mathbb{F}2\mathbb{Z}$  ([Construction 3.1](#)) is to reduce the claim

$$\prod_{i \in [\ell_1]} ((g^{\gamma_i} - 1) \cdot \pi_2(\mathbf{f}_{ij}) + 1) = g^{\mu_j} \in \mathbb{F}_{2^n}, \quad \text{for all } j \in [\ell_2]. \quad (4)$$

to  $\ell_2$  inner product claims for the vectors  $\pi_2(\mathbf{f}_{*j})$ . We do this reduction via the GKR protocol for multiple grand products, and make several implementation-level optimizations for it.

[Albert: Todo](#)

### 3.7 Proof size optimization for large inputs

When  $\ell = \ell_1 \cdot \ell_2$  is such that  $\ell_2$  is large, say  $\ell \geq 2^{11}$ , the elements  $\mu_1, \dots, \mu_{\ell_2} \in \mathbb{Z}$  sent at Step 1 of [Construction 3.1](#) may start to dominate the proof size. For example, for  $\ell = 2^{13}$  and  $\mu_i$  of 128 bits, the elements  $\mu_1, \dots, \mu_{\ell_2}$  amount to 131 KB. To mitigate this cost, the idea is to have the prover commit to the vector  $(\mu_1, \dots, \mu_{\ell_2})$  and then prove (1), i.e.

$$\sum_{j \in [\ell_1]} \pi_q(\mu_j) \cdot \mathbf{v}_j^{(2)} = \mu \quad \text{over } \mathbb{F}_q.$$

using an IOPP for constrained codes. Concretely, the prover commits to the vector  $\text{bits}(\mu)$  of bits forming the elements  $(\mu_1, \dots, \mu_{\ell_2})$  using  $\mathbb{F}2\mathbb{Z}$ , and then proves (1) using [Construction 3.1](#). At the end of [Construction 3.1](#), the prover is left with an inner product claim for the vector  $\pi_2(\text{bits}(\mu))$ . [Albert: TODO: explain how to batch this claim with the WHIR IOPP for the final step of Construction 3.1](#)

## 4 Virtualization of $\mathbb{F}_2$ -linear combinations lifted to integers

In this section we expand relation  $\mathcal{R}_{\mathcal{C}, \mathbb{F}_{2^n}, \mathbb{F}_q, \text{Pack}_n}^{\text{lin-dual}}$  to impose, instead of constraints of the form  $\text{Enc}_{\mathcal{C}}(\text{Pack}_n(\pi_2(\mathbf{f}))) = f$  and  $\langle \pi_q(\mathbf{f}), \mathbf{v} \rangle = \mu$ , constraints as  $\text{Enc}_{\mathcal{C}}(\text{Pack}_n(\pi_2(\mathbf{f}))) = f$  and

$$\langle \pi_q(\pi_2^{-1}(M \cdot \pi_2(\mathbf{f}))) \rangle = \mu$$

for a matrix  $M \in \mathbb{F}_2^{\ell' \times \ell}$ . Precisely, given a new parameter  $\ell'$ , we define

$$\mathcal{R}_{\mathcal{C}, \mathbb{F}_{2^n}, \mathbb{F}_q, \text{Pack}_n}^{\text{lin-virtual}} := \left\{ \left( \begin{array}{l} \mathbb{x} = (\mu, \mathbf{v}, M) \in \mathbb{F}_q \times \mathbb{F}_q^{\ell'} \times \mathbb{F}_2^{\ell' \times \ell}, \\ \mathbb{y} = f \in \mathbb{F}_{2^n}^m, \\ \mathbb{w} = \mathbf{f} \in \{0, 1\}^\ell \subseteq \mathbb{Z}^\ell \end{array} \right) \mid \begin{array}{l} \text{Enc}_{\mathcal{C}}(\text{Pack}_n(\pi_2(\mathbf{f}))) = f \\ \wedge \langle \pi_q(\pi_2^{-1}(M \cdot \pi_2(\mathbf{f}))), \mathbf{v} \rangle = \mu \end{array} \right\}$$

**Albert:** We could replace  $M \cdot \pi_2(\mathbf{f})$  by the output of a GKR circuit on  $\pi_2(\mathbf{f})$  The relation has the tag “lin(ear)-virtual” to reference the intuition that  $\mathcal{R}_{\mathcal{C}, \mathbb{F}_{2^n}, \mathbb{F}_q, \text{Pack}_n}^{\text{lin-virtual}}$  expresses a linear constraint on the vector  $\mathbf{h} = \pi_q(\pi_2^{-1}(M \cdot \pi_2(\mathbf{f}))) \in \mathbb{F}_q^{\ell'}$ , which is not committed directly: only  $\pi_2(\mathbf{f})$  is. Note that  $\mathbf{h}$  is a linear expression over  $\mathbb{F}_{2^n}$  of the elements in  $\pi_2(\mathbf{f})$ , lifted to the integers, and then projected to  $\mathbb{F}_q$ .

**Albert:** Would be good to add more detail in this paragraph The relation  $\mathcal{R}_{\mathcal{C}, \mathbb{F}_{2^n}, \mathbb{F}_q}^{\text{lin-virtual}}$  allows to, for example, express linear constraints over  $\mathbb{F}_q$  of the XOR of witness elements, while only committing (over  $\mathbb{F}_{2^n}$ ) to the original elements. The same applies with other bit operations such as AND and Majority, by noting first that, given integer bits  $x, y, z \in \{0, 1\}$ :

**Albert:** Should reference some identity in preliminaries, or explain more here

$$\begin{aligned} \pi_q(x \text{ AND } y) &= 2^{-1} \cdot \pi_q(x + y - \pi_2^{-1}(\pi_2(x) + \pi_2(y))), \\ \pi_q(\text{Maj}(x, y, z)) &= 2^{-1} \cdot \pi_q(x + y + z - \pi_2^{-1}(\pi_2(x) + \pi_2(y) + \pi_2(z))), \end{aligned}$$

where  $2^{-1}$  denotes the inverse of 2 modulo  $q$ . Hence AND and Majority can be expressed over  $\mathbb{F}_q$  as linear expressions of a vectors of the form  $\pi_q(\pi_2^{-1}(M \cdot \pi_2(\mathbf{f})))$ , with  $\mathbf{f}$  being a vector of integer bits containing only the original elements on which we apply the AND and Majority operations.

We next provide an IOPP for  $\mathcal{R}_{\mathcal{C}, \mathbb{F}_{2^n}, \mathbb{F}_q, \text{Pack}_n}^{\text{lin-virtual}}$  for specific shapes of  $M$ . Similarly as in ??, we assume a tensor structure  $\ell = \ell_1 \cdot \ell_2$  for  $\mathbf{v}$ , and write  $\mathbf{f} = (\mathbf{f}_{*j})_{j \in [\ell_2]} \in \{0, 1\}^{\ell_1 \times \ell_2}$ . We then let  $\ell'_1$  be such that  $\ell' = \ell'_1 \times \ell_2$  and assume  $M = \text{Id} \otimes M_0$  where  $M_0$  is a  $\ell'_1 \times \ell_1$  matrix with entries in  $\mathbb{F}_2$ , and  $\text{Id}$  is the  $\ell_2 \times \ell_2$  matrix. In words,  $M$  is the  $\ell' \times \ell = \ell'_1 \cdot \ell_2 \times \ell_1 \cdot \ell_2$  diagonal block matrix with  $M_0$  as diagonal blocks. Then

$$M \cdot \pi_2(\mathbf{f}) = (M_0 \cdot \pi_2(\mathbf{f}_{*j}))_{j \in [\ell_2]} \in (\mathbb{F}_2^{\ell'_1})^{\ell_2}. \quad (5)$$

**Construction 4.1** (IOPP for  $\mathcal{R}_{\mathcal{C}, \mathbb{F}_{2^n}, \mathbb{F}_q}^{\text{lin-virtual}}$ ). The prover and verifier receive  $\mathbf{x} = (\mu, \mathbf{v}, M) \in \mathbb{F}_q \times \mathbb{F}_q^{\ell'} \times \mathbb{F}_2^{\ell' \times \ell}$  and  $\mathbf{y} = f \in \mathbb{F}_{2^n}^m$ . The prover receives  $\mathbf{w} = \mathbf{f} \in \{0, 1\}^\ell \subseteq \mathbb{Z}^\ell$  such that  $(\mathbf{x}, \mathbf{y}, \mathbf{w}) \in \mathcal{R}_{\mathcal{C}, \mathbb{F}_{2^n}, \mathbb{F}_q}^{\text{lin-virtual}}$ . We assume  $M$  is of the form described above.

Define  $\mathbf{h} = \pi_2^{-1}(M \cdot \pi_2(f)) \in \{0, 1\}^{\ell'} \subseteq \mathbb{Z}^{\ell'}$ . By (5),

$$\mathbf{h} = (\mathbf{h}_{*j})_{j \in [\ell_2]} = (\pi_2^{-1}(M_0 \cdot \pi_2(\mathbf{f}_{*j})))_{j \in [\ell_2]},$$

with  $\mathbf{h}_{*j} \in \{0, 1\}^{\ell'_1}$ . Let  $f = \text{Enc}_{\mathcal{C}}(\text{Pack}_n(\pi_2(\mathbf{f}))) \in \{0, 1\}^m \subseteq \mathbb{Z}^m$ .

1. Prover and verifier execute [Construction 3.1](#) with inputs

$$\mathbf{x}' = (\mu, \mathbf{v}), \quad \mathbf{y}' = f, \quad \mathbf{w}' = \mathbf{h} = \pi_2^{-1}(M \cdot \pi_2(\mathbf{f})) \in \{0, 1\}^{\ell'} \subseteq \mathbb{Z}^{\ell'}.$$

until Step 3 where they are meant to use the IOR  $\Pi_{\text{red}}$ . Note that until that point, the verifier has not made any queries to  $\mathbf{y}'$ . So, in terms of completeness, it does not matter, yet, that  $f$  is not the encoding of  $\pi_2(\mathbf{h}) = M \cdot \pi_2(\mathbf{f})$ .

2. Prover and verifier engage in  $\Pi_{\text{red}}^{\ell_2}$  with inputs

$$\left( \mathbf{x}_j = ((g^{\gamma_i})_{i \in [\ell'_1]}, g^{\mu_j}), \mathbf{y}_j = f_{*j}, \mathbf{w}_j = \pi_2(\mathbf{h}_{*j}) = M_0 \cdot \pi_2(\mathbf{f}_{*j}) \right)_{j \in [\ell_2]},$$

where  $f_{*j} = \text{Enc}_{\mathcal{C}_1}(\text{Pack}_n(\pi_2(\mathbf{f}_{*j})))$ . At the end of  $\Pi_{\text{red}}^{\ell_2}$  they are left with claims

$$((\mathbf{v}, \eta_j), \mathbf{y}_j, \mathbf{w}_j)_{j \in [\ell_2]} \in \left( \mathcal{R}_{\mathcal{C}_1, \mathbb{F}_{2^n}, \text{Pack}_n}^{\text{lin}} \right)^{\ell_2}, \quad (6)$$

for some  $\mathbf{v} \in \mathbb{F}_{2^n}^{\ell'_1}$  and  $\eta_j \in \mathbb{F}_{2^n}$ . Again, since we assume the verifier never queries  $\mathbf{y}_j$  during  $\Pi_{\text{red}}$ , completeness holds for honest provers, even if  $\mathbf{y}_j$  is not the encoding of  $\mathbf{w}_j$ .

Note that (6) holds if and only if, for all  $j \in [\ell_2]$ ,

$$\begin{aligned} \eta_j &= \sum_{i \in [\ell'_1]} \mathbf{v}_i \cdot \pi_2(\mathbf{h}_{ij}) = \sum_{i \in [\ell'_1]} \mathbf{v}_i \cdot \sum_{t \in [\ell_1]} M_{0it} \cdot \pi_2(\mathbf{f}_{tj}) \\ &= \sum_{t \in [\ell_1]} \left( \sum_{i \in [\ell'_1]} M_{0it} \cdot \mathbf{v}_i \right) \cdot \pi_2(\mathbf{f}_{tj}) = \langle \mathbf{u}_j, \pi_2(\mathbf{f}_{*j}) \rangle, \end{aligned}$$

where  $\mathbf{u} \in \mathbb{F}_{2^n}^{\ell'_1}$  is the vector whose  $t$ -th entry is

$$\mathbf{u}_t = \langle \mathbf{v}, M_{0*t} \rangle \in \mathbb{F}_{2^n},$$

and  $M_{0*t} \in \mathbb{F}_{2^n}^{\ell'_1}$  denotes the  $t$ -th column of  $M_0$ , for  $t \in [\ell_1]$ ,

Note that, if  $\Pi_{\text{red}}^{\ell_2}$  is as in ??, then  $\mathbf{v} = (\text{eq}(\mathbf{b}, \mathbf{r}))_{\mathbf{b} \in \{0, 1\}^{\log \ell'_1}}$  for some  $\mathbf{r} \in \mathbb{F}_{2^n}^{\log \ell'_1}$ , and then  $\mathbf{u}_t = \text{mle}[M_{0*t}](\mathbf{r}) \in \mathbb{F}_{2^n}$ .

3. If the prover is honest, we have that

$$((\mathbf{u}, \eta_j), \mathbb{Y}_j = \text{Enc}_{\mathcal{C}_1}(\text{Pack}_n(\pi_2(\mathbf{f}_{*j}))), \pi_2(\mathbf{f}_{*j}))_{j \in [\ell_2]} \in \left( \mathcal{R}_{\mathcal{C}_1, \mathbb{F}_{2^n}, \text{Pack}_n}^{\text{lin}} \right)^{\ell_2}. \quad (7)$$

Prover and verifier use the Galois ring-switch [Construction A.1](#) to reduce (7) to a claim of the form.

$$((\mathbf{u}', \kappa_j), \mathbb{Y}_j = \text{Enc}_{\mathcal{C}_1}(\text{Pack}_n(\pi_2(\mathbf{f}_{*j}))), \text{Pack}_n(\pi_2(\mathbf{f}_{*j})))_{j \in [\ell_2]} \in \left( \mathcal{R}_{\mathcal{C}_1, \mathbb{F}_{2^n}, \text{id}}^{\text{lin}} \right)^{\ell_2}, \quad (8)$$

where  $\mathbf{u}' = (\text{eq}(\mathbf{b}, \mathbf{r}))_{\mathbf{b} \in \{0,1\}^{\log \ell_1}}$  for some  $\mathbf{r} \in \mathbb{F}_{2^n}^{\log \ell_1}$ , and for some  $\kappa_j \in \mathbb{F}_{2^n}$ .

4. Note that (8) is a collection of standard claims about the MLE evaluation of a vector with entries in  $\mathbb{F}_{2^n}$ , “committed” through an encoding over  $\mathbb{F}_{2^n}$ . Accordingly, prover and verifier use an IOPP to prove such claims. [Albert: To keep in mind: ring-switch for arbitrary inner products could be expensive](#)

**Example 4.2.** Say we have a vector  $\mathbf{f} \in \{0,1\}^\ell \subseteq \mathbb{Z}^\ell$  and we wish to prove MLE evaluation claims of an extended vector  $\mathbf{f}' \in \{0,1\}^{2\ell}$  where

$$\mathbf{f}'_i = \begin{cases} \mathbf{f}_i & \text{if } i \in [\ell], \\ \mathbf{f}_{i-\ell} \text{ XOR } \mathbf{f}_{i-\ell+1} & \text{if } \ell+1 \leq i \leq 2 \cdot \ell - 1, \\ \mathbf{f}_\ell & \text{if } i = 2 \cdot \ell. \end{cases}$$

In words,  $\mathbf{f}'$  is the result of concatenating  $\mathbf{f}$  and the vector obtained by “xor-ing” together every two consecutive elements in  $\mathbf{f}$  (and keeping the last entry as  $\mathbf{f}_\ell$ , since  $\mathbf{f}_\ell \text{ XOR } \mathbf{f}_{\ell+1}$  is not defined). This example will come up in our SHA-256 case-study.

We have

$$\mathbf{f}' = \pi_2^{-1}(M \cdot \pi_2(\mathbf{f})) \in \{0,1\}^{2\ell}$$

where

$$M = \begin{pmatrix} & & & & & \\ & & & & & \\ & & \text{Id}_{\ell \times \ell} & & & \\ \hline 1 & 1 & 0 & 0 & \dots & 0 \\ 0 & 1 & 1 & 0 & \dots & 0 \\ \vdots & & \ddots & & & \vdots \\ 0 & \dots & 0 & 0 & 1 & 1 \\ 0 & \dots & 0 & 0 & 0 & 1 \end{pmatrix}$$

if we do not wish to leverage any tensor decomposition structure, i.e. if we take  $\ell_1 = \ell$  and  $\ell_2 = 1$ , then we can apply [Construction 4.1](#) using this matrix  $M$  as part of the input.

At the beginning of Step 3 of [Construction 4.1](#), prover and verifier obtain the claim

$$((\mathbf{u}, \eta), \text{Enc}_C(\text{Pack}_n(\pi_2(\mathbf{f}))), \pi_2(\mathbf{f})) \in \mathcal{R}_{\mathcal{C}_1, \mathbb{F}_{2^n}, \text{Pack}_n}^{\text{lin}},$$

for

$$\mathbf{u} = (\text{mle}[M_{*t}](\mathbf{r}))_{t \in [\ell]} \in \mathbb{F}_{2^n}^\ell,$$

where  $M_{*t}$  denotes the  $t$ -th column of  $M$ . Then, prover and verifier use the Galois ring-switch [Construction A.1](#) to reduce the above claim to standard MLE evaluation claims of  $\text{Pack}_n(\pi_2(\mathbf{f}_{*j}))$ . The cost of doing so for the vector  $\mathbf{u}$  of linear coefficients is .... [Albert: discuss](#).

Next we address the case when we use a non-trivial tensor decomposition structure  $\ell = \ell_1 \cdot \ell_2$  with  $\ell_1 \neq 1 \neq \ell_2$ . [Albert: TODO](#)

**Theorem 4.3.** [Albert: Completeness and RBR knowledge soundness of Construction 3.1](#)

*Proof.* □

#### 4.1 Optimizing $\mathbb{F}_2\mathbb{Z}$ for virtualized inputs $\pi_2^{-1}(M \cdot \pi_2(\mathbf{f}))$

In [Construction 4.1](#), at Step 2, prover and verifier use  $\Pi_{\text{red}}^{\ell_2}$  to reduce the claim

$$\prod_{i \in [\ell_1]} ((g^{\gamma_i} - 1) \cdot (M_0 \cdot \pi_2(\mathbf{f}_{*j}))_i + 1) = g^{\mu_j} \quad \in \mathbb{F}_{2^n}, \quad \text{for all } j \in [\ell_2] \quad (9)$$

to  $\ell_2$  inner product claims over  $\mathbb{F}_{2^n}$  for the vectors  $M_0 \cdot \pi_2(\mathbf{f}_{*j})$ . In many applications of interest (see ??), the vectors  $M_0 \cdot \pi_2(\mathbf{f}_{*j})$  are larger than  $\mathbf{f}_{*j}$ , but contain entries that are structured expressions of the entries in  $\mathbf{f}_{*j}$ . For example, in [Example 4.2](#),  $M \cdot \pi_2(\mathbf{f})$  is the concatenation of  $\pi_2(\mathbf{f})$  and the vector with entries  $\pi_2(\mathbf{f})_i \text{ XOR } \pi_2(\mathbf{f})_{i+1}$ .

A natural question is whether, in such scenarios, the proof of (9) can be optimized so that it costs less than a full proof for  $\ell_2$  arbitrary input vectors of the same length as  $M_0 \cdot \pi_2(\mathbf{f}_{*j})$ . In this section we show that this is indeed possible.

## 5 R1CS constraints over polynomial rings with ideal membership

[Albert: I'm aiming to make the paper self-contained, so I will refer to Zinc/Zinc+ but will provide all required definitions and proofs in this paper](#)

## 6 Open questions for us

Albert: Regarding Section 4.1, I've only found a way to cheapen the following settings: say first and two vectors in the grand product are  $\mathbf{g}_1, \mathbf{g}_2$ , and third is  $\mathbf{g}_1 \text{ XOR } \mathbf{g}_2$ . This is a tricky problem. We should look more into it

Albert: Should we allow for  $W > 1$ ? Currently just setting  $W = 1$  and thinking of all bits as unrelated leads to better performance, but that sounds suboptimal.

## 7 Case-study

Albert: World passport? Lean circuits? ML models? Hashes?

### 7.1 Integer and modular multiplication

Albert: We could do something synthetic, like a chain of mults and additions, or a ML or Lean motivated computation

### 7.2 SHA-256 hashing

**SHA-256 compression overview** We begin with a brief overview of SHA-256 to establish notation, then formalize the relation we seek to arithmetize.

For the purposes of SHA-256, a *word* is a  $\leq 32$  string of bits, which we represent as a binary integer polynomial, i.e. as an element of  $\{0, 1\}^{<32}[X]$ . The SHA-256 compression function receives as input a 512-bit string, represented as sixteen *message words*  $(M_t)_{t \in [16]} \in \{0, 1\}^{<32}[X]^{16}$ . The algorithm also uses 64 fixed *round constants*  $(K_t)_{t \in [64]} \in \{0, 1\}^{<32}[X]^{64}$ .

We let

$$\begin{aligned} \psi_2 : \mathbb{Z}[X] &\rightarrow \mathbb{Z} \\ a(X) &\mapsto a(2) \end{aligned} \tag{10}$$

be the ring homomorphism sending integer polynomials to their evaluation at  $X = 2$ . Notice that, given  $a \in \{0, 1\}^{<32}[X]$ , the integer  $\psi_2(a) \in \mathbb{Z}$  is the integer whose bit-representation coincides with the coefficients of  $a$ .

SHA-256 is built from the following six auxiliary functions. For  $x, y, z \in \{0, 1\}^{<32}[X]$ ,



define:

$$\begin{aligned}
\Sigma_0(x) &:= \text{ROTR}^2(x) \text{ XOR } \text{ROTR}^{13}(x) \text{ XOR } \text{ROTR}^{22}(x) && \in \{0, 1\}^{<32}[X] \\
\Sigma_1(x) &:= \text{ROTR}^6(x) \text{ XOR } \text{ROTR}^{11}(x) \text{ XOR } \text{ROTR}^{25}(x) && \in \{0, 1\}^{<32}[X] \\
\text{Ch}(x, y, z) &:= (x \text{ AND } y) \text{ XOR } (\neg x \text{ AND } z) && \in \{0, 1\}^{<32}[X] \\
\text{Maj}(x, y, z) &:= (x \text{ AND } y) \text{ XOR } (x \text{ AND } z) \text{ XOR } (y \text{ AND } z) && \in \{0, 1\}^{<32}[X] \\
\sigma_0(x) &:= \text{ROTR}^7(x) \text{ XOR } \text{ROTR}^{18}(x) \text{ XOR } \text{SHR}^3(x) && \in \{0, 1\}^{<32}[X] \\
\sigma_1(x) &:= \text{ROTR}^{17}(x) \text{ XOR } \text{ROTR}^{19}(x) \text{ XOR } \text{SHR}^{10}(x) && \in \{0, 1\}^{<32}[X]
\end{aligned} \tag{11}$$

Albert: todo: define the bit operations

**Message schedule.** At the beginning of the SHA-256 compression function, the sixteen input message words  $(M_t)_{t \in [16]}$  are expanded into a so-called *message schedule*, which consists of 64 integers of 32 bits  $(W_t)_{t \in [64]} \in \mathbb{Z}^{64}$  defined as follows:

$$W_t := \begin{cases} \psi_2(M_t) & t \in [16], \\ \psi_2(W_{t-16}) + \psi_2(\sigma_0(W_{t-15})) + \psi_2(W_{t-7}) + \psi_2(\sigma_1(W_{t-2})) \pmod{2^{32}} & t \in \{17, \dots, 64\}. \end{cases} \tag{12}$$

**Compression loop.** After the message schedule is computed, SHA-256 performs the so-called *compression loop*. Throughout it, the function maintains an 8-word internal state, conventionally labeled as

$$(a, b, c, d, e, f, g, h) \in \{0, 1\}^{<32}[X]^8.$$

The compression loop iterates 64 rounds. At each round  $t \in [64]$ , the state entering that round is denoted  $(a_t, b_t, c_t, d_t, e_t, f_t, g_t, h_t) \in \{0, 1\}^{<32}[X]^8$ . The per-round update is given by, for  $t \in [64]$ ,

$$\begin{aligned}
T_{1,t} &:= \psi_2(h_t) + \psi_2(\Sigma_1(e_t)) + \psi_2(\text{Ch}(e_t, f_t, g_t)) + \psi_2(K_t) + W_t \pmod{2^{32}} && \in \mathbb{Z}, \\
T_{2,t} &:= \psi_2(\Sigma_0(a_t)) + \psi_2(\text{Maj}(a_t, b_t, c_t)) \pmod{2^{32}} && \in \mathbb{Z}, \\
a_{t+1} &:= \psi_2^{-1}(T_{1,t} + T_{2,t} \pmod{2^{32}}) && \in \{0, 1\}^{<32}[X], \\
e_{t+1} &:= \psi_2^{-1}(\psi_2(d_t) + \psi_2(T_{1,t}) \pmod{2^{32}}) && \in \{0, 1\}^{<32}[X], \\
(b_{t+1}, c_{t+1}, d_{t+1}, f_{t+1}, g_{t+1}, h_{t+1}) &:= (a_t, b_t, c_t, e_t, f_t, g_t) && \in \{0, 1\}^{<32}[X]^6.
\end{aligned}$$

The full SHA-256 compression function adds the initial state to the terminal state word-wise modulo  $2^{32}$  (the so-called *feed-forward step*); we omit this step for brevity [Albert: Let's not do that](#). For our purposes, the output of the compression function is the final state  $(a_{65}, \dots, h_{65})$ .

## Our arithmetization

### 7.3 Experiments

Albert: GHASH or b127?

Optimization ideas

- b127 instead of GHASH
- Lookup table (algorithm-wise, not SNARK-wise) for first levels of GKR
- If we have an MLE claim of a vector and its shift by one position, can we reuse something for the latter claim from the computations made when proving the former?

#### 7.3.1 Comparison with Hyrax

Albert: Comparing with Hyrax may be more interesting than it sounds initially. Hyrax is pay-per-bit and opens on large prime fields: a profile that F2Z can adopt. Hence (maybe) projects using Hyrax could use F2Z as a drop-in replacement which is PQ secure and more efficient

#### 7.3.2 Comparison with Flock

John: It would be interesting to compare with flock as well

## A Ring switching via Galois orbits

Albert: Rewriting the previous appendix for arbitrary inner products

**Construction A.1.** [Galois ring-switch: an IOR from  $\mathcal{R}_{\mathcal{C}, \mathbb{F}_{2^n}, \text{Pack}_n}^{\text{lin}}$  to  $\mathcal{R}_{\mathcal{C}_1, \mathbb{F}_{2^n}, \text{id}}^{\text{mle}}$ ] **Albert:** **TODO:** formulate as a reduction from  $\mathcal{R}_{\mathcal{C}, \mathbb{F}_{2^n}, \text{Pack}_n}^{\text{lin}}$  to  $\mathcal{R}_{\mathcal{C}_1, \mathbb{F}_{2^n}, \text{id}}^{\text{mle}}$ .

Let  $b_1, \dots, b_m \in \mathbb{F}_2^{2^n}$ . Using an  $\mathbb{F}_2$ -basis  $\beta_1, \dots, \beta_m$  of  $\mathbb{F}_{2^m}$  we pack  $m$  inner products

$$\langle b_i, \mathbf{v} \rangle = \sum_{j \in \{0,1\}^n} b_{i,j} \cdot \mathbf{v}_j, \quad i = 1, \dots, m,$$

coefficient-wise into a single inner product

$$\begin{aligned} b &:= b_1 \cdot \beta_1 + \dots b_m \cdot \beta_m \in \mathbb{F}_{2^m}^{2^n} \\ \langle b, \mathbf{v} \rangle &= \langle b_1, \mathbf{v} \rangle \cdot \beta_1 + \dots + \langle b_m, \mathbf{v} \rangle \cdot \beta_m \\ &= \sum_{j \in \{0,1\}^n} (b_{1,j} \cdot \beta_1 + \dots + b_{m,j} \cdot \beta_m) \cdot \mathbf{v}_j. \end{aligned}$$

The Frobenius automorphism  $\phi(z) = z^2$  keeps  $\mathbb{F}_2 \subset \mathbb{F}_{2^m}$  fixed, and has no other fixed points. We extract the value of  $\langle b_i, \mathbf{v} \rangle$  from the values of  $\langle b, \mathbf{v} \rangle$  along the Galois orbit

$$\mathbf{v}, \mathbf{v}^2, \mathbf{v}^{2^2}, \dots, \mathbf{v}^{2^{m-1}},$$

as follows, where by  $\mathbf{v}^t$  we mean that each component of  $\mathbf{v}$  is raised to the power  $t$ . The Moore matrix

$$M = \begin{pmatrix} \beta_1 & \beta_2 & \dots & \beta_m \\ \beta_1^2 & \beta_2^2 & \dots & \beta_m^2 \\ \beta_1^{2^2} & \beta_2^{2^2} & \dots & \beta_m^{2^2} \\ \vdots & \vdots & \ddots & \vdots \\ \beta_1^{2^{m-1}} & \beta_2^{2^{m-1}} & \dots & \beta_m^{2^{m-1}} \end{pmatrix}$$

consists of the Galois orbits of the basis vectors, and is invertible. (See below.) Using that the component polynomials are over  $\mathbb{F}_2$ , we see that

$$\phi^k \left( \langle b, \phi^{-k}(\mathbf{v}) \rangle \right) = \langle b_1, \mathbf{v} \rangle \cdot \phi^k \beta_1 + \dots + \langle b_m, \mathbf{v} \rangle \cdot \phi^k \beta_m,$$

for any  $k$ . To this end, observe that

$$M \cdot \begin{pmatrix} \langle b_1, \mathbf{v} \rangle \\ \langle b_2, \mathbf{v} \rangle \\ \langle b_3, \mathbf{v} \rangle \\ \vdots \\ \langle b_m, \mathbf{v} \rangle \end{pmatrix} = \begin{pmatrix} \langle b, \mathbf{v} \rangle \\ \phi \left( \langle b, \phi^{-1}(\mathbf{v}) \rangle \right) \\ \phi^2 \left( \langle b, \phi^{-2}(\mathbf{v}) \rangle \right) \\ \vdots \\ \phi^{m-1} \left( \langle b, \phi^{-(m-1)}(\mathbf{v}) \rangle \right) \end{pmatrix}.$$

Multiplying both sides by  $M^{-1}$  we obtain an expression of the values  $\langle b_i, \mathbf{v} \rangle$  as a linear expression of the values  $\phi^k \left( \langle b, \phi^{-k}(\mathbf{v}) \rangle \right)$ . **Ulrich:** Invertibility of  $M$  follows from that of  $M^t$ : any linear functional  $(c_1, \dots, c_m)$  in the kernel of  $M^t$  gives rise to the linearizable polynomial

$$p(X) = \sum_{i=0}^{m-1} c_i \cdot X^{2^i}$$

which has  $\beta_1, \dots, \beta_m$ , and hence the whole space  $F$  as roots. By dimension,  $p(X)$  must be trivial, meaning that all its coefficients are zero.

## B Bilinear Embeddings

Given a field  $\mathbb{F}$  and a degree- $d$  extension  $\mathbb{E}$ . Our goal is to commit to a vector  $\mathbf{w} \in \mathbb{F}^{nd}$  so that we can later use it to prove an inner product claim of the form  $\langle \mathbf{w}, \mathbf{a} \rangle_{\mathbb{F}} = h$  for some  $\mathbf{a} \in \mathbb{F}^{nd}$  and  $h \in \mathbb{F}$ . However, we only have an efficient inner product commitment scheme to commit to vectors  $\tilde{\mathbf{w}} \in \mathbb{E}^n$  and open as  $\langle \tilde{\mathbf{w}}, \tilde{\mathbf{a}} \rangle_{\mathbb{E}} = \tilde{h}$  for some  $\tilde{\mathbf{a}} \in \mathbb{E}^n$  and  $\tilde{h} \in \mathbb{E}$ .

To do this, we seek a triple of maps  $(W, A, H)$  that satisfy:

$$\langle \mathbf{w}, \mathbf{a} \rangle_{\mathbb{F}} = H \left( \langle W(\mathbf{w}), A(\mathbf{a}) \rangle_{\mathbb{E}} \right).$$

## B.1 Background

### B.1.1 Perfect pairings

In the following let  $\mathbb{E}/\mathbb{F}$  be a finite field extension of degree  $d$ . If we disregard multiplication in  $\mathbb{E}$ , then  $\mathbb{E}$  is a  $d$ -dimensional vector space over  $\mathbb{F}$ , hence isomorphic to  $\mathbb{F}^d$ . Let  $\mathbf{e}_1, \dots, \mathbf{e}_d$  denote the standard basis of  $\mathbb{F}^d$ . Recall that a function  $\mu : \mathbb{E} \times \mathbb{E} \rightarrow \mathbb{F}$  is a *bilinear form* if it is linear in each argument separately. It is *symmetric* if  $\mu(x, y) = \mu(y, x)$  for all  $x, y \in \mathbb{E}$ . For each  $x \in \mathbb{E}$ , the partial application in the first argument defines a linear form

$$\mu_x : \mathbb{E} \rightarrow \mathbb{F} \quad y \mapsto \mu(x, y).$$

Thus  $\mu_x \in \mathbb{E}^*$  where  $\mathbb{E}^* = \text{Hom}_{\mathbb{F}}(\mathbb{E}, \mathbb{F})$  denotes the dual vector space. The bilinear form therefore induces an  $\mathbb{F}$ -linear map

$$\Phi_\mu : \mathbb{E} \rightarrow \mathbb{E}^* \quad x \mapsto \mu_x.$$

We call  $\mu$  *nondegenerate*, or equivalently a *perfect pairing*, if  $\Phi_\mu$  is an isomorphism. Or equivalently for finite spaces, [Albert: add a comment that injectivity implies surjectivity when the vector spaces have finite dimension?](#)

$$\mu(x, y) = 0 \text{ for all } y \in \mathbb{E} \implies x = 0.$$

Since  $\mu$  is a perfect pairing, every basis  $(\beta_1, \dots, \beta_d)$  of  $\mathbb{E}$  has a unique dual basis  $(\beta_1^\vee, \dots, \beta_d^\vee)$  satisfying  $\mu(\beta_i, \beta_j^\vee) = \delta_{ij}$ .

*John: It would be good to explain how you would typically derive  $\beta_j^\vee$*

### B.1.2 Succinct vectors

Given a field  $\mathbb{F}$ , a vector  $\mathbf{a} \in \mathbb{F}^n$  for  $n = 2^k$  is called *succinct* if it admits a tensor decomposition

$$\begin{bmatrix} m_{00} \\ m_{01} \end{bmatrix} \otimes \begin{bmatrix} m_{10} \\ m_{11} \end{bmatrix} \otimes \dots \otimes \begin{bmatrix} m_{(k-1)0} \\ m_{(k-1)1} \end{bmatrix} \quad (13)$$

where typically  $m_{ij} \in \mathbb{F}$ . The multilinear extension of  $\mathbf{a}$ , viewed as a function on  $\{0, 1\}^k$ , then becomes

$$\tilde{a}(\mathbf{r}) = \prod_{i=0}^{k-1} ((1 - r_i)m_{i0} + r_i m_{i1}). \quad (14)$$

which can be evaluated in  $O(k)$  field operations.

More generally, the coefficients  $m_{ij}$  need not themselves be field elements. It suffices that they belong to an associative  $\mathbb{F}$ -algebra in which every complete product  $m_{0b_0} m_{1b_1} \dots m_{(k-1)b_{k-1}}$  evaluates to an element of  $\mathbb{F}$ . Consequently, every vector entry and every value of the multilinear extension remains an element of  $\mathbb{F}$ . As the  $m_{ij}$  no longer need to be commutative we must therefore also fix a permutation of the variables  $\mathbf{r}$ .

Since every finite-dimensional associative algebra admits a matrix representation, we may, without loss of generality, assume that the objects  $m_{ij}$  are matrices. In fact, it suffices to take  $m_{ij} \in \mathbb{F}^{w_i \times w_{i+1}}$  with boundary dimensions  $w_0 = w_k = 1$ . We call  $\max w_i$  the *width* of the representation, which is known as a *matrix branching program* and is closely related to matrix product states and weighted finite automata.

The canonical example of a succinct vector is the multilinear evaluation vector  $a_i = \text{eq}(\mathbf{r}, \text{bits}(i))$  obtained by taking  $m_{i0} = 1 - r_i$  and  $m_{i1} = r_i$ . Another classical example is the geometric progression  $(1, x, x^2, \dots)$  obtained by taking  $m_{i0} = 1$  and  $m_{i1} = x^{2^i}$ . An illustrative non-scalar example is the counting vector  $(0, 1, 2, \dots)$  represented by

$$m_{0b} = \begin{bmatrix} 2 & b \end{bmatrix} \quad m_{ib} = \begin{bmatrix} 2 & b \\ 0 & 1 \end{bmatrix} \quad m_{(k-1)b} = \begin{bmatrix} b \\ 1 \end{bmatrix}.$$

## B.2 Structure

We are now equipped to prove a theorem on the structure of embeddings. Under the assumption that the output map is  $\mathbb{F}$ -linear, we show that for  $n = 1$  the space of possible embeddings is very structured.

**Theorem B.1** (Structure of bilinear embeddings). *Let  $W, A : \mathbb{F}^d \rightarrow \mathbb{E}, H : \mathbb{E} \rightarrow \mathbb{F}$  be maps such that for all  $\mathbf{w}, \mathbf{a} \in \mathbb{F}^d$ ,*

$$\langle \mathbf{w}, \mathbf{a} \rangle_{\mathbb{F}} = H(W(\mathbf{w})A(\mathbf{a})).$$

*Suppose that  $H$  is  $\mathbb{F}$ -linear. Then:*

1.  *$W$  and  $A$  are  $\mathbb{F}$ -linear isomorphisms,*
2.  *$\mu_H(x, y) = H(xy)$  is a perfect pairing,*
3.  *$(W(\mathbf{e}_1), \dots, W(\mathbf{e}_d))$  and  $(A(\mathbf{e}_1), \dots, A(\mathbf{e}_d))$  are dual bases with respect to  $\mu_H$ .*

*Conversely, if  $W$  and  $A$ , and  $H$  are  $\mathbb{F}$ -linear,  $\mu_H$  is a perfect pairing and the images of the standard basis under  $W$  and  $A$  are dual bases with respect to  $\mu_H$ , then the identity*

$$\langle \mathbf{w}, \mathbf{a} \rangle_{\mathbb{F}} = H(W(\mathbf{w})A(\mathbf{a})).$$

*holds for all  $\mathbf{w}, \mathbf{a} \in \mathbb{F}^d$ .*

*Proof of the forward direction.* First, observe that  $\mu_H(W(\mathbf{e}_i), A(\mathbf{e}_j)) = \langle \mathbf{e}_i, \mathbf{e}_j \rangle_{\mathbb{F}} = \delta_{ij}$ . Taking  $i = j$  shows  $H$  is nonzero. Since  $H$  is  $\mathbb{F}$ -linear,  $\mu_H$  is a symmetric bilinear form, we show that it is nondegenerate. Let  $x \in \mathbb{E}$  be nonzero. Since multiplication by  $x$  is a bijection of  $\mathbb{E}$  and  $H$  is nonzero, there exists  $y \in \mathbb{E}$  such that  $\mu_H(x, y) = H(xy) \neq 0$ . Thus  $\mu_H$  is nondegenerate, and hence a perfect pairing.

Next, we show that  $W$  and  $A$  are bijective. Suppose  $W(\mathbf{w}_1) = W(\mathbf{w}_2)$ . Then for every  $\mathbf{a} \in \mathbb{F}^d$ ,

$$\langle \mathbf{w}_1 - \mathbf{w}_2, \mathbf{a} \rangle_{\mathbb{F}} = \mu_H(W(\mathbf{w}_1) - W(\mathbf{w}_2), A(\mathbf{a})) = 0.$$

By nondegeneracy of the standard inner product on  $\mathbb{F}^d$ , this implies  $\mathbf{w}_1 = \mathbf{w}_2$ . Hence  $W$  is injective. Since  $\mathbb{F}^d$  and  $\mathbb{E}$  have the same finite cardinality,  $W$  is bijective. The same argument shows that  $A$  is bijective.

We now prove that  $W$  is  $\mathbb{F}$ -linear. For  $\mathbf{w}_1, \mathbf{w}_2, \mathbf{a} \in \mathbb{F}^d$ , linearity of  $H$  gives

$$\begin{aligned} \mu_H(W(\mathbf{w}_1 + \mathbf{w}_2) - W(\mathbf{w}_1) - W(\mathbf{w}_2), A(\mathbf{a})) \\ = \langle \mathbf{w}_1 + \mathbf{w}_2, \mathbf{a} \rangle_{\mathbb{F}} - \langle \mathbf{w}_1, \mathbf{a} \rangle_{\mathbb{F}} - \langle \mathbf{w}_2, \mathbf{a} \rangle_{\mathbb{F}} \\ = 0. \end{aligned}$$

Since  $A$  is surjective, the second argument ranges over all of  $\mathbb{E}$ . Nondegeneracy of  $\mu_H$  therefore implies  $W(\mathbf{w}_1 + \mathbf{w}_2) = W(\mathbf{w}_1) + W(\mathbf{w}_2)$ . Similarly, for  $\lambda \in \mathbb{F}$ ,

$$\begin{aligned} \mu_H(W(\lambda \mathbf{w}) - \lambda W(\mathbf{w}), A(\mathbf{a})) \\ = \langle \lambda \mathbf{w}, \mathbf{a} \rangle_{\mathbb{F}} - \lambda \langle \mathbf{w}, \mathbf{a} \rangle_{\mathbb{F}} \\ = 0. \end{aligned}$$

Again using surjectivity of  $A$  and nondegeneracy of  $\mu_H$ , we obtain  $W(\lambda \mathbf{w}) = \lambda W(\mathbf{w})$ . Hence  $W$  is  $\mathbb{F}$ -linear. By symmetry,  $A$  is also  $\mathbb{F}$ -linear. Since they are also bijective,  $W$  and  $A$  are linear isomorphisms. From this it follows that  $(W(\mathbf{e}_1), \dots, W(\mathbf{e}_d))$  and  $(A(\mathbf{e}_1), \dots, A(\mathbf{e}_d))$  are bases of  $\mathbb{E}$ . Finally, from  $\mu_H(W(\mathbf{e}_i), A(\mathbf{e}_j)) = \delta_{ij}$  it follows they are dual with respect to  $\mu_H$ .  $\square$

*Proof of the converse direction.* Can be directly shown using a basis expansion. Write  $\mathbf{w} = \sum_{i=1}^d w_i \mathbf{e}_i$  and  $\mathbf{a} = \sum_{j=1}^d a_j \mathbf{e}_j$ . Since  $W$  and  $A$  are  $\mathbb{F}$ -linear,  $W(\mathbf{w}) = \sum_{i=1}^d w_i W(\mathbf{e}_i)$  and  $A(\mathbf{a}) = \sum_{j=1}^d a_j A(\mathbf{e}_j)$ . Therefore,

$$\begin{aligned} H(W(\mathbf{w})A(\mathbf{a})) &= \mu_H(W(\mathbf{w}), A(\mathbf{a})) \\ &= \sum_{i,j} w_i a_j \mu_H(W(\mathbf{e}_i), A(\mathbf{e}_j)) \\ &= \sum_{i,j} w_i a_j \delta_{ij} \\ &= \sum_i w_i a_i \\ &= \langle \mathbf{w}, \mathbf{a} \rangle_{\mathbb{F}}. \end{aligned}$$

$\square$

The  $n = 1$  case extends naturally to  $n > 1$  by doing blockwise packing, i.e. we embed using  $W' = I_n \otimes W$  and  $A' = I_n \otimes A$  and keep the same  $H$ . Let  $\mathbf{w} = (\mathbf{w}_1, \dots, \mathbf{w}_n)$  and

$\mathbf{a} = (\mathbf{a}_1, \dots, \mathbf{a}_n)$  then

$$\begin{aligned}
H(\langle W'(\mathbf{w}), A'(\mathbf{a}) \rangle_{\mathbb{E}}) &= H\left(\sum_{i=1}^n W(\mathbf{w}_i)A(\mathbf{a}_i)\right) \\
&= \sum_{i=1}^n H(W(\mathbf{w}_i)A(\mathbf{a}_i)) \\
&= \sum_{i=1}^n \langle \mathbf{w}_i, \mathbf{a}_i \rangle_{\mathbb{F}} \\
&= \langle \mathbf{w}, \mathbf{a} \rangle_{\mathbb{F}}.
\end{aligned}$$

Going forward, we use  $W$  and  $A$  also for their blockwise extensions when the meaning is clear. We do not consider embeddings that mix coordinates across different blocks of size  $d$ , as such embeddings are unlikely to be useful in practice.

We have shown that every  $n = 1$  embedding with a linear output map is determined by a choice of a nonzero linear map  $H$  and linear isomorphism  $W$ . The map  $W$  then determines the first basis  $\beta_i = W(\mathbf{e}_i)$  and the map  $A$  is then forced to map to the  $\mu_H$ -dual basis.

The linearity assumption on the output map is necessary. Indeed, composing a linear embedding with a multiplicative bijection  $g : \mathbb{E} \rightarrow \mathbb{E}$  generally produces an embedding that is nonlinear. Whether every nonlinear embedding arises from such a multiplicative reparameterization is a separate question, which we do not need here.

An important property of bilinear embeddings is that they preserve succinctness if the block size aligns with the tensor decomposition:

**Theorem B.2** (Preservation of succinctness). *Given  $d = 2^k$ ,  $n = 2^l$  and a  $\mathbb{F}$ -linear blockwise map  $A = I_n \otimes A_0$  for some isomorphism  $A_0 : \mathbb{F}^d \rightarrow \mathbb{E}$ , then if  $\mathbf{a}$  is succinct then so is  $A(\mathbf{a})$ . In particular if*

$$\mathbf{a} = \bigotimes_{i=0}^{k+l-1} \begin{bmatrix} m_{i0} \\ m_{i1} \end{bmatrix}$$

where the  $m_{ij}$  can be scalars or  $w_i \times w_{i+1}$  matrices, then

$$A(\mathbf{a}) = \left( \bigotimes_{i=0}^{l-1} \begin{bmatrix} m_{i0} \\ m_{i1} \end{bmatrix} \right) \cdot (I_{w_l} \otimes A_0) \left( \bigotimes_{i=l}^{l+k-1} \begin{bmatrix} m_{i0} \\ m_{i1} \end{bmatrix} \right).$$

where the second factor is column vector in  $\mathbb{E}^{w_l}$  which can be absorbed into the matrices  $m_{(l-1)b}$ , bringing the representation back into canonical form.

### B.3 Constructions

The set of nonzero linear output maps  $H$  is  $\mathbb{E}^* \setminus \{0\}$  which is isomorphic to  $\mathbb{E}^\times$ . The set of linear isomorphism  $W$  is isomorphic to  $\text{GL}_d(\mathbb{F})$ , hence the total number of bilinear embeddings is  $|\mathbb{E}^\times| \cdot |\text{GL}_d(\mathbb{F})|$ . We'll now look at specific choices and their properties.

#### B.3.1 Trace embedding

Let  $q = |\mathbb{F}|$ , the  $q$ -power Frobenius map is  $\varphi : \mathbb{E} \rightarrow \mathbb{E}$  given by  $x \mapsto x^q$ . It is an  $\mathbb{F}$ -linear field automorphism of order  $d$ , and its fixed field is precisely  $\mathbb{F}$ :

$$\mathbb{E}^\varphi = \{x \in \mathbb{E} : \varphi(x) = x\} = \mathbb{F}.$$

The field trace is the  $\mathbb{F}$ -linear map  $\text{Tr}_{\mathbb{E}/\mathbb{F}} : \mathbb{E} \rightarrow \mathbb{F}$  defined by

$$\text{Tr}_{\mathbb{E}/\mathbb{F}}(x) = \sum_{i=0}^{d-1} \varphi^i(x) = \sum_{i=0}^{d-1} x^{q^i}.$$

To see that this sum is indeed in  $\mathbb{F} \subseteq \mathbb{E}$  observe the sum is fixed by  $\varphi$ , since

$$\varphi\left(\sum_{i=0}^{d-1} \varphi^i(x)\right) = \sum_{i=1}^d \varphi^i(x) = \sum_{i=0}^{d-1} \varphi^i(x),$$

where  $\varphi^d(x) = \varphi^0(x)$ . Hence the trace lies in the fixed field  $\mathbb{F}$ .

This makes  $\text{Tr}_{\mathbb{E}/\mathbb{F}}$  a natural choice of output map  $H$ . More generally, every  $\mathbb{F}$ -linear map  $H : \mathbb{E} \rightarrow \mathbb{F}$  is uniquely of the form  $H(x) = \text{Tr}_{\mathbb{E}/\mathbb{F}}(\alpha x)$  for some  $\alpha \in \mathbb{E}$ . Consequently, the nonzero output maps are parameterized by  $\mathbb{E}^\times$ .

The Galois orbit embedding chooses the basis  $\beta_i = \varphi^i(\gamma)$  which has dual  $\beta_i^\vee = \varphi^i(\gamma^\vee)$ .

Ulrich: This sounds to me very much related to Galois Fribinius on an inner product with a general covector. You would end up with the multi-claim of the inner products of the packed poly with the Galois orbit of the covector. This is how I view it: Given  $\mathbf{p}_1, \dots, \mathbf{p}_d \in \mathbb{F}^n$ , and  $\mathbf{q}_1, \dots, \mathbf{q}_d \in \mathbb{F}^n$ , we are interested in

$$\sum_{k=1}^d \langle \mathbf{p}_k, \mathbf{q}_k \rangle,$$

right?

The packing map sends the coeff vectors  $\mathbf{p}_1, \dots, \mathbf{p}_d$  to  $\mathbf{p} = \beta_1 \cdot \mathbf{p}_1 + \dots + \beta_d \cdot \mathbf{p}_d$ , where  $\beta_1, \dots, \beta_d$  is an  $\mathbb{F}$ -basis of  $\mathbb{E}$ . Choose the element  $a_k$  in  $\mathbb{E}$  which extracts the  $k$ -th coordinate

$$\mathbf{p}_k = \Phi_k(\mathbf{p}) = \text{Tr}_{\mathbb{E}/\mathbb{F}}(a_k \cdot \mathbf{p}).$$



Then

$$\begin{aligned}\langle \mathbf{p}_k, \mathbf{q}_k \rangle &= \left\langle \sum_{i=0}^d \varphi^i(a_k \cdot \mathbf{p}), \mathbf{q}_k \right\rangle \\ &= \sum_{i=0}^d \varphi^i(a_k) \cdot \varphi^i(\langle \mathbf{p}, \mathbf{q}_k \rangle),\end{aligned}$$

since  $\mathbf{q}_k$  is over  $\mathbb{F}$ . This reduces our inner product claim to that of the packed poly  $\mathbf{p}$  with the unpacked polys  $\mathbf{q}_1, \dots, \mathbf{q}_k$ .

But does that match our use case? We are rather interested in  $\langle \mathbf{p}_k, \mathbf{v} \rangle$  for some  $\mathbf{v} \in \mathbb{E}^n$ . In this case,

$$\begin{aligned}\langle \mathbf{p}_k, \mathbf{v} \rangle &= \left\langle \sum_{i=0}^d \varphi^i(a_k \cdot \mathbf{p}), \mathbf{q}_k \right\rangle \\ &= \sum_{i=0}^d \varphi^i(a_k) \cdot \varphi^i(\langle \mathbf{p}, \varphi^{-i}(\mathbf{v}) \rangle),\end{aligned}$$

and we end up with inner products of the packed poly with the Galois orbit of  $\mathbf{v}$ .

### B.3.2 Coefficient projection

In the coefficient projection we assume a concrete representation of  $\mathbb{E}$  as  $\mathbb{F}[X]/(f(X))$  for some monic degree  $d$  polynomial  $f$  irreducible over  $\mathbb{F}$ . We assume  $f$  is chosen for implementation efficiency: it is sparse with mostly low order terms. Further we represent members of  $\mathbb{E}$  in the monomial basis  $(1, X, X^2, \dots, X^{d-1})$ . We can then represent  $W$  and  $A$  by invertible matrices in  $\mathbb{F}^{d \times d}$  and  $H$  by a vector in  $\mathbb{F}^d$ . For implementation efficiency we seek an embedding with sparse  $W$  and  $A$ . It turns out taking the most efficient choices for  $H$  and  $W$  also gives efficient  $A$ :

**Theorem B.3.** *Write  $f(X) = f_d X^d + f_{d-1} X^{d-1} + \dots + f_1 X + f_0$  so that  $f_d = 1$  and  $f_0 \neq 0$ . Take  $W = I_d$  and  $H = (1, 0, \dots, 0)$ . Then the dual-basis embedding  $A$  is given by*

$$A_{ij} = \begin{cases} 1, & i = j = 0, \\ -f_{i+j}/f_0, & i, j \geq 1 \text{ and } i + j \leq d, \\ 0, & \text{otherwise.} \end{cases}$$

In particular,  $A$  is a bordered upper anti-triangular Hankel matrix. For  $d = 5$  it looks like

$$A = \frac{1}{f_0} \begin{pmatrix} f_0 & 0 & 0 & 0 & 0 \\ 0 & -f_2 & -f_3 & -f_4 & -1 \\ 0 & -f_3 & -f_4 & -1 & 0 \\ 0 & -f_4 & -1 & 0 & 0 \\ 0 & -1 & 0 & 0 & 0 \end{pmatrix}.$$

Notably  $f_1$  does not appear and for  $k > 1$  the  $f_k$  coefficient appears  $k - 1$  times. This perfectly matches the usual implementation preference for sparse reduction polynomials with mostly low-order terms.

For the 127-bit field  $\mathbb{F}_2[X]/(X^{127} + X + 1)$  the  $A$  map is simply a reversal of the  $i > 0$  coordinates. For the GHASH field  $\mathbb{F}_2[X]/(X^{128} + X^7 + X^2 + X + 1)$  the map additionally needs seven XOR corrections.

## B.4 Extension openings

Assume now that we wish to open a base-field witness  $\mathbf{w} \in \mathbb{F}^{nd}$  against extension-field weights  $\mathbf{a} \in \mathbb{E}^{nd}$ , proving the claim  $\langle \mathbf{w}, \mathbf{a} \rangle_{\mathbb{E}} = h$  for some  $h \in \mathbb{E}$ . We present a one-round batching protocol and a  $\log d$ -round folding protocol.

### B.4.1 Batching protocol

Given a bilinear embedding  $(W, A, H)$  and a basis  $(\beta_1, \dots, \beta_d)$  for  $\mathbb{E}/\mathbb{F}$ , we reduce the extension-field claim by decomposing into basis coordinates and batching the resulting claims with a random linear combination.

**Construction B.4** (Batching protocol for extension opening). Given the claim  $\langle \mathbf{w}, \mathbf{a} \rangle_{\mathbb{E}} = h$ , define  $\mathbf{a}_i \in \mathbb{F}^{nd}$  such that

$$\mathbf{a} = \sum_{i=1}^d \mathbf{a}_i \beta_i.$$

The parties proceed as follows.

1. The prover sends  $h_i = \langle W(\mathbf{w}), A(\mathbf{a}_i) \rangle_{\mathbb{E}}$  for  $i = 1, \dots, d$ .
2. The verifier checks  $h = \sum_{i=1}^d H(h_i) \beta_i$ .
3. The verifier samples an  $\epsilon$ -zero-evader  $\boldsymbol{\rho} \leftarrow_{\$} \mathbb{E}^d$ .
4. The parties reduce the claim to  $\langle W(\mathbf{w}), \mathbf{a}' \rangle_{\mathbb{E}} = h'$  where

$$h' = \sum_{i=1}^d \rho_i h_i, \quad \mathbf{a}' = \sum_{i=1}^d \rho_i A(\mathbf{a}_i).$$

This interactive oracle reduction is always complete, has round-by-round soundness error  $\epsilon$ , and proof size  $d \log |\mathbb{E}| = d^2 \log |\mathbb{F}|$ .

**Theorem B.5** (Succinctness of coordinate decomposition). *Given a succinct vector  $\mathbf{a} \in \mathbb{E}^{nd}$  for  $d = 2^k$  and a basis  $(\beta_i)$  of  $\mathbb{E}/\mathbb{F}$  then the  $\beta$ -coordinate decomposed vectors are also succinct. In particular if*

$$\mathbf{a} = \bigotimes_{i=0}^{k+l-1} \begin{bmatrix} m_{i0} \\ m_{i1} \end{bmatrix}$$

where the  $m_{ij}$  can be scalars or  $w_i \times w_{i+1}$  matrices, then

$$\mathbf{a}_i = \mathbf{u}_i \cdot \left( \bigotimes_{j=0}^{k+l-1} \begin{bmatrix} \Phi(m_{j0}) \\ \Phi(m_{j1}) \end{bmatrix} \right) \cdot \mathbf{v}$$

where  $\Phi : \mathbb{E} \rightarrow \mathbb{F}^{d \times d}$  is the map that represents an  $\mathbb{E}$ -element  $m$  by the matrix representing multiplication by  $m$  in basis  $(\beta_i)$ ,  $\mathbf{u}_i$  is the one-hot vector with  $u_{ij} = \delta_{ij}$ , and  $\mathbf{v}$  is the base  $(\beta_i)$  representation of  $1 \in \mathbb{E}$ .  $\Phi$  is applied elementwise to  $w_i \times w_{i+1}$  matrices over  $\mathbb{E}$  to produce  $dw_i \times dw_{i+1}$  block matrices over  $\mathbb{F}$ . Absorbing the outer vectors in the matrices brings the representation back into canonical form, with the width increased by at most a factor  $d$ .

For practical implementations it makes sense to evaluate the branching program over  $\mathbb{E}$  directly and then extract the coordinates in the basis  $(\beta_i)$ . This is equivalent to omitting the output selector  $\mathbf{t}_i$  to keep the output dimension  $w_0 = d$ .

If  $A = I_n \otimes A_0$  is blockwise we can also apply theorem B.2 and compose it with the combination weights  $\boldsymbol{\rho} \in \mathbb{E}^d$  to get a succinct representation for  $\mathbf{a}'$ :

$$\mathbf{a}' = \boldsymbol{\rho} \cdot \left( \bigotimes_{i=0}^{l-1} \begin{bmatrix} \Phi(m_{i0}) \\ \Phi(m_{i1}) \end{bmatrix} \right) \cdot (I_{dw_l} \otimes A_0) \left( \bigotimes_{i=l}^{l+k-1} \begin{bmatrix} \Phi(m_{i0}) \\ \Phi(m_{i1}) \end{bmatrix} \right) \cdot \mathbf{v}. \quad (15)$$

The map  $\Phi$  is an algebra embedding, so replacing each scalar  $m_{ij} \in \mathbb{E}$  by  $\Phi(m_{ij})$  leaves the branching-program computation unchanged, except that the values are now represented in the basis  $(\beta_i)$  representation of  $\mathbb{E}$ . The only operations outside the embedded  $\mathbb{E}$  computation are the blockwise embedding  $A_0$  and the random linear combination  $\boldsymbol{\rho}$ . Thus, apart from the applications of  $A_0$  and  $\boldsymbol{\rho}$ , evaluating  $\tilde{\mathbf{a}}'$  is the same computation as evaluating  $\tilde{\mathbf{a}}$ , performed in the regular representation of  $\mathbb{E}$ . **Remco: Something is off with the complexity. Should really check with a Sage implementation. Probably needs a factor  $d$ .**

The batch protocol is quadratic in  $d$  in terms of  $\mathbb{F}$ -element operations and proof size. This is acceptable when  $d$  is small, but binary fields where  $d = 128$  the communication would be 2,048 bytes. Avoiding this quadratic dependence on  $d$  appears to require further rounds of interaction and more sophisticated decomposition.

#### B.4.2 Folding protocol

In the folding protocol we interactively reduce the claim  $\langle \mathbf{w}, \mathbf{a} \rangle_{\mathbb{E}} = h$  through a sequence of embeddings  $W_i$  where at round  $i$  the claim is

$$\langle W_i(\mathbf{w}), A_i(\mathbf{a}) \rangle_{\mathbb{E}} = h_i \quad W_i : \mathbb{F}^{nd} \rightarrow \mathbb{E}^{nd_i} \quad A_i : \mathbb{E}^{nd} \rightarrow \mathbb{E}^{nd_i}$$

Here  $d = d_0 \geq \dots \geq d_k = 1$  is a series of folding dimensions. We start with  $W_0$  and  $A_0$  the identity and  $h_0 = h$  so that the initial claim is indeed  $\langle \mathbf{w}, \mathbf{a} \rangle_{\mathbb{E}} = h$ . We also stipulate that

the embeddings are blockwise such that  $W_i = \mathbf{I}_n \otimes W'_i$  and  $A_i = \mathbf{I}_n \otimes A'_i$ . For simplicity we will only consider the  $n = 1$  case from here on as  $n > 1$  follows by blockwise repetition.

The final map is  $W_k : \mathbb{F}^d \rightarrow \mathbb{E}$  which is an  $F$ -linear isomorphism. Let  $(\beta_i)$  be the implied basis such that

$$W_k(\mathbf{x}) = \sum_{i=1}^d x_i \beta_i.$$

Working backwards from the final embedding we create a sequence

$$R_i \circ W_{i+1} = W_i$$

where each map  $R_i$  has only  $\left\lceil \frac{d_i}{d_{i+1}} \right\rceil$  Frobenius components.

Every  $\mathbb{F}$ -linear map  $T : \mathbb{E}^m \rightarrow \mathbb{E}^n$  can be viewed as being built from  $\mathbb{E}$ -linear maps and  $\mathbb{F}$ -linear scalar operations on  $\mathbb{E}$ . To recursively fold such maps, the scalar operations must be compatible with arbitrary  $\mathbb{E}$ -linear coordinate transformations. Concretely, we require an intertwining

$$\beta(R\mathbf{x}) = \alpha(R)\beta(\mathbf{x}),$$

where  $\alpha$  acts entrywise on  $R$ . This property singles out semilinear scalar operators. Over finite fields, these are precisely the Frobenius maps  $\sigma^i$  up to scalar multiples, and the Frobenius powers therefore provide the canonical scalar decomposition.

**Theorem B.6** (Frobenius decomposition). *Let  $\mathbb{E}/\mathbb{F}$  be a finite extension of degree  $d$ , and let  $\sigma : \mathbb{E} \rightarrow \mathbb{E}$  be the Frobenius map  $x \mapsto x^{|\mathbb{F}|}$ . Then every  $\mathbb{F}$ -linear map  $T : \mathbb{E}^m \rightarrow \mathbb{E}^n$  admits a unique decomposition*

$$T(\mathbf{x}) = \sum_{i=0}^{d-1} M_i \cdot (\mathbf{I}_m \otimes \sigma^i)(\mathbf{x}), \quad M_i \in \mathbb{E}^{n \times m}.$$

We define  $\text{supp}_\sigma(T) := \{i : M_i \neq 0\}$ .

**Single folding round.** Assume the current claim is

$$\langle W_i(\mathbf{w}), A_i(\mathbf{a}) \rangle_{\mathbb{E}} = h_i.$$

Let

$$R_i : \mathbb{E}^{nd_{i+1}} \rightarrow \mathbb{E}^{nd_i}$$

be a fixed  $\mathbb{F}$ -linear map satisfying

$$R_i \circ W_{i+1} = W_i.$$

By the Frobenius decomposition there exist unique matrices

$$M_{i,0}, \dots, M_{i,d-1} \in \mathbb{E}^{nd_i \times nd_{i+1}}$$

such that

$$R_i = \sum_{j=0}^{d-1} M_{i,j} \circ (\mathbf{I}_{nd_{i+1}} \otimes \sigma^j).$$

For every  $j$ , define the  $\mathbb{F}$ -linear map

$$C_{i,j} = (\mathbf{I}_{nd_{i+1}} \otimes \sigma^{-j}) \circ M_{i,j}^\top.$$

Then, for every  $\mathbf{x} \in \mathbb{E}^{nd_{i+1}}$  and  $\mathbf{b} \in \mathbb{E}^{nd_i}$ ,

$$\langle R_i(\mathbf{x}), \mathbf{b} \rangle_{\mathbb{E}} = \sum_{j=0}^{d-1} \sigma^j(\langle \mathbf{x}, C_{i,j}(\mathbf{b}) \rangle_{\mathbb{E}}).$$

Applying this identity to

$$\mathbf{x} = W_{i+1}(\mathbf{w}), \quad \mathbf{b} = A_i(\mathbf{a}),$$

gives

$$\langle W_i(\mathbf{w}), A_i(\mathbf{a}) \rangle_{\mathbb{E}} = \sum_{j=0}^{d-1} \sigma^j(\langle W_{i+1}(\mathbf{w}), C_{i,j}(A_i(\mathbf{a})) \rangle_{\mathbb{E}}).$$

The prover sends the values

$$g_{i,j} = \langle W_{i+1}(\mathbf{w}), C_{i,j}(A_i(\mathbf{a})) \rangle_{\mathbb{E}}, \quad j = 0, \dots, d-1.$$

The verifier checks

$$h_i = \sum_{j=0}^{d-1} \sigma^j(g_{i,j}),$$

samples a random challenge  $\rho_i \in \mathbb{E}$ , and defines

$$A_{i+1} = \left( \sum_{j=0}^{d-1} \rho_i^j C_{i,j} \right) \circ A_i.$$

Finally, letting

$$h_{i+1} = \sum_{j=0}^{d-1} \rho_i^j g_{i,j},$$

the next claim is

$$\langle W_{i+1}(\mathbf{w}), A_{i+1}(\mathbf{a}) \rangle_{\mathbb{E}} = h_{i+1}.$$

We therefore seek an interactive reduction from a mixed claim to a packed claim

$$\begin{aligned} \langle \mathbf{w}, \mathbf{a} \rangle_{\mathbb{E}} &= h & \mathbf{w} &\in \mathbb{F}^{nd} & \mathbf{a} &\in \mathbb{E}^{nd} \\ \Downarrow & & & & & \\ \langle W(\mathbf{w}), \mathbf{a}' \rangle_{\mathbb{E}} &= h' & W(\mathbf{w}) &\in \mathbb{E}^n & \mathbf{a}' &\in \mathbb{E}^n \end{aligned}$$

where  $W : \mathbb{F}^{nd} \rightarrow \mathbb{E}^n$  is a fixed  $\mathbb{F}$ -linear embedding and  $\mathbf{a}' \in \mathbb{E}^n$ ,  $h' \in \mathbb{E}$  are determined by  $\mathbf{a}$ ,  $h$  and the transcript. Ideally  $\mathbf{a} \mapsto \mathbf{a}'$  should also preserve useful properties like efficient computation of its multilinear extension.

Above we decomposed  $\mathbf{a}$  with respect to a basis of  $\mathbb{E}/\mathbb{F}$ . Here we instead decompose the induced  $\mathbb{F}$ -linear functional on packed witnesses. Define

$$T_{\mathbf{a}} : \mathbb{E}^n \rightarrow \mathbb{E} \quad \mathbf{x} \mapsto \langle W^{-1}(\mathbf{x}), \mathbf{a} \rangle_{\mathbb{E}}.$$

This is an  $\mathbb{F}$ -linear functional on  $\mathbb{E}^n$ , but is not generally  $\mathbb{E}$ -linear. We'll first prove a representation lemma for  $\mathbb{F}$ -linear  $\mathbb{E} \rightarrow \mathbb{E}$  maps and then a decomposition theorem.

**Theorem B.7.** *Let  $\mathbb{F} \subseteq \mathbb{E}$  be finite fields with  $q = |\mathbb{F}|$  and  $[\mathbb{E} : \mathbb{F}] = d$ , then every  $\mathbb{F}$ -linear map  $T : \mathbb{E} \rightarrow \mathbb{E}$  is uniquely represented by a  $q$ -linearized polynomial*

$$P(X) = \sum_{j=0}^{d-1} c_j X^{q^j} \quad c_j \in \mathbb{E}.$$

*Proof.* By interpolation every map  $T : \mathbb{E} \rightarrow \mathbb{E}$  can be represented uniquely by a polynomial  $P \in \mathbb{E}[X]$  of degree less than  $|\mathbb{E}| = q^d$

$$P(X) = \sum_{j=0}^{q^d-1} a_j X^j \quad a_j \in \mathbb{E}.$$

From  $\mathbb{F}$ -linearity follows  $P(0) = 0$  and hence  $a_0 = 0$ , further we have  $P(X + Y) = P(X) + P(Y)$  and hence

$$\begin{aligned} 0 &= P(X + Y) - P(X) - P(Y) \\ &= \sum_{j=1}^{q^d-1} a_j (X + Y)^j - \sum_{j=1}^{q^d-1} a_j (X^j + Y^j) \\ &= \sum_{j=1}^{q^d-1} a_j \sum_{k=0}^j \binom{j}{k} X^k Y^{j-k} - \sum_{j=1}^{q^d-1} a_j (X^j + Y^j) \\ &= \sum_{j=1}^{q^d-1} a_j \sum_{k=1}^{j-1} \binom{j}{k} X^k Y^{j-k} \end{aligned}$$

Write  $q = p^r$  with  $p$  the characteristic of  $\mathbb{F}$ . For every  $j$  with  $a_j \neq 0$  we must have  $\binom{j}{k} = 0 \pmod p$  for all  $0 < k < j$ , by Lucas's theorem this holds iff  $j = p^s$  for some  $s$ . Hence we can reindex  $P$  keeping only the non-zero coefficients

$$P(X) = \sum_{j=0}^{rd-1} b_j X^{p^j} \quad b_j \in \mathbb{E}.$$

From  $\mathbb{F}$ -linearity also follows  $P(\lambda x) = \lambda P(x)$  for all  $x \in \mathbb{E}$  and  $\lambda \in \mathbb{F}$ . For every  $b_j \neq 0$  this requires  $\lambda^{p^j} = \lambda$  for every  $\lambda \in \mathbb{F}$ . This holds exactly when  $j = rs$  for some  $s$ . Reindexing  $P$  again we get our claimed result

$$P(X) = \sum_{j=0}^{d-1} c_j X^{q^j} \quad c_j \in \mathbb{E}.$$

□

**Theorem B.8** (Frobenius decomposition). *Let  $\mathbb{F} \subseteq \mathbb{K} \subseteq \mathbb{E}$  be finite fields with  $[\mathbb{K} : \mathbb{F}] = m$ , and let  $\sigma(x) = x^{|\mathbb{F}|}$  denote the  $\mathbb{F}$ -Frobenius automorphism. Then every  $\mathbb{F}$ -linear map  $T : \mathbb{E} \rightarrow \mathbb{E}$  admits a unique decomposition*

$$T(x) = \sum_{i=0}^{m-1} \sigma^i \circ T_i$$

where each  $T_i$  is  $\mathbb{K}$ -linear.

*Proof.* Write  $q = |\mathbb{F}|$  and  $d = [\mathbb{E} : \mathbb{F}]$ , by the preceding theorem  $T$  has a unique representation

$$T(X) = \sum_{j=0}^{d-1} c_j X^{q^j} \quad c_j \in \mathbb{E}.$$

Since  $\mathbb{K} \subseteq \mathbb{E}$ , we have  $m \mid d$ . Write each index uniquely as  $j = ms + i$ , with  $0 \leq i < m$  and  $0 \leq s < d/m$ . Grouping the terms by their residue modulo  $m$  gives

$$T(X) = \sum_{i=0}^{m-1} \sum_{s=0}^{d/m-1} c_{ms+i} X^{q^{ms+i}}.$$

For each residue class  $i$ , define

$$T_i(X) = \sum_{s=0}^{d/m-1} c_{ms+i}^{q^{-i}} X^{q^{ms}},$$

where  $z \mapsto z^{q^{-i}}$  denotes the inverse of the automorphism  $z \mapsto z^{q^i}$  of  $\mathbb{E}$ . Each  $T_i$  is now a  $q^m$ -linearized polynomial hence applying the preceding theorem to the extension  $\mathbb{K} \subseteq \mathbb{E}$ , each  $T_i$  is  $\mathbb{K}$ -linear. Then

$$(\sigma^i \circ T_i)(X) = T_i(X)^{q^i} = \sum_{s=0}^{d/m-1} c_{ms+i} X^{q^{ms+i}}.$$

Therefore  $T$  represented as

$$T = \sum_{i=0}^{m-1} \sigma^i \circ T_i.$$

Uniqueness follows immediately from the uniqueness of the  $q$ -linear polynomial representation.  $\square$

The decomposition theorem extends coordinatewise to maps  $\mathbb{E}^n \rightarrow \mathbb{E}$ . In particular, the functional  $T_{\mathbf{a}}(\mathbf{x}) = \langle W^{-1}(\mathbf{x}), \mathbf{a} \rangle_{\mathbb{E}}$  admits a unique decomposition

$$T_{\mathbf{a}} = \sum_{i=0}^{m-1} \sigma^i \circ T_i, \tag{16}$$



where each  $T_i : \mathbb{E}^n \rightarrow \mathbb{E}$  is  $\mathbb{K}$ -linear. Thus the initial claim may be written as

$$h = T_{\mathbf{a}}(W(\mathbf{w})) = \sum_{i=0}^{m-1} \sigma^i(T_i(W(\mathbf{w}))).$$

**Construction B.9** (Frobenius fold). Given the claim  $T_{\mathbf{a}}(W(\mathbf{w})) = h$ , the parties proceed as follows.

1. The prover sends  $v_i = T_i(W(\mathbf{w}))$  for  $i = 1, \dots, m-1$ .
2. The verifier derives  $v_0 = h - \sum_{i=1}^{m-1} \sigma^i(v_i)$ .
3. The verifier samples independent challenges  $\rho_1, \dots, \rho_{m-1} \leftarrow \$ \mathbb{E}$ .
4. The parties reduce to the claim  $T'(W(\mathbf{w})) = h'$ , where

$$T' = T_0 + \sum_{i=1}^{m-1} \rho_i T_i \quad \text{and} \quad h' = v_0 + \sum_{i=1}^{m-1} \rho_i v_i.$$

**Construction B.10** (Folding protocol for extension opening). Choose a tower of intermediate fields  $\mathbb{F} = \mathbb{K}_0 \subseteq \mathbb{K}_1 \subseteq \dots \subseteq \mathbb{K}_t = \mathbb{E}$ , and write  $m_i = [\mathbb{K}_{i+1} : \mathbb{K}_i]$ . Initially,  $T_{\mathbf{a}}$  is  $\mathbb{K}_0$ -linear. At round  $i$ , apply the Frobenius fold with  $\mathbb{F} = \mathbb{K}_i$ , and  $\mathbb{K} = \mathbb{K}_{i+1}$ . This reduces the current  $\mathbb{K}_i$ -linear functional to a  $\mathbb{K}_{i+1}$ -linear functional.

After  $t$  rounds, the residual functional  $T' : \mathbb{E}^n \rightarrow \mathbb{E}$  is  $\mathbb{E}$ -linear. Consequently, there exists  $\mathbf{a}' \in \mathbb{E}^n$  such that  $T'(\mathbf{x}) = \langle \mathbf{x}, \mathbf{a}' \rangle_{\mathbb{E}}$ . The final claim is therefore  $\langle W(\mathbf{w}), \mathbf{a}' \rangle_{\mathbb{E}} = h'$ , as required.

Each round requires sending  $m_i - 1$  elements of  $\mathbb{E}$ . For a binary tower where  $m_i = 2$ ,  $\mathbb{F} = \mathbb{F}_2$ , and  $\mathbb{E} = \mathbb{F}_{2^{128}}$  we have 7 rounds of one  $\mathbb{E}$  element message each, for a total of 112 bytes. Compare to the non-interactive protocol this reduces the communication complexity from  $O(d^2)$  to  $O(d \log d)$  elements of  $\mathbb{F}_2$ .

**Towerless folding.** Say  $\mathbb{F} \subseteq \mathbb{E}$  admits no tower of subfields, for example  $\mathbb{E} = \mathbb{F}_{2^{127}}$ . Take again  $d = [\mathbb{E} : \mathbb{F}]$  and decompose  $T_{\mathbf{a}}$  as

$$T_{\mathbf{a}} = \sum_{i=0}^{d-1} \sigma^i \circ T_i,$$

We can still recursively fold using the protocol above with  $m = 2$  by grouping even and odd exponents of  $\sigma^i$  (zero-padding as necessary), as there is no actual dependence on the subfield.

This folding progressively halves  $d$  (rounding up) in the  $\mathbb{E}^{nd}$  inner product. This induces a sequence of perfect pairings

$$(\mathbb{F}^{nd}, \mathbb{E}^{nd}) \rightsquigarrow (\mathbb{E}^{nd/m}, \mathbb{E}^{nd/m}) \rightsquigarrow (\mathbb{K}_i^{nd/m}, \mathbb{E}^{nd/m})$$

$$\langle W_i(\mathbf{w}), A_i(\mathbf{a}) \rangle = \langle \mathbf{w}, \mathbf{a} \rangle.$$

The final result is a transcript dependent map  $A : \mathbb{E}^{dn} \rightarrow \mathbb{E}^n$ . If  $W$  is blockwise with  $I_n \otimes W_0$  for some  $W_0 : \mathbb{F}^d \rightarrow \mathbb{E}$  then so is  $A$  with  $A_0 : \mathbb{E}^d \rightarrow \mathbb{E}$ .

## C GKR protocol for multiple grand products